Natural Language Processing

An Introduction

Abdelbacet Mhamdi

2026-04-04

MT @ ISET Bizerte

Outline

1. Terminology 2
2. Introduction to Regular Expressions (Regex) 5
3. Text Tokenization 35
4. Text Processing and Visualization 70
5. Word Embedding 113
6. Deep Learning for NLP 128

1. Terminology

1. Terminology

Natural Language Processing (NLP) is a subfield of **AI** that focuses on the interaction between computers and linguistics. It involves the development of algorithms and models that enable machines to understand, interpret, and generate human language in a way that is both meaningful and useful.

Examples: *Text analysis, sentiment analysis, machine translation, chatbots.*

A **corpus** is a large, structured collection of texts used for linguistic analysis and model training. **Corpora** provide diverse language samples that help algorithms learn patterns, relationships, and semantic meanings.

Examples: *Wikipedia dumps, news articles, twitter data.*

1. Terminology

A **vocabulary** is the complete set of unique words or tokens recognized by a language model. The **vocabulary** defines the model's linguistic boundaries and is typically extracted from training data with frequency thresholds.

🔥 **Tip**

dictionary, word list, token collection.

A **document** refers to a discrete text unit processed as a single entity. **Documents** can vary greatly in length and structure but serve as the fundamental unit for many text analysis tasks.

Examples: *emails, articles, tweets, books.*

2. Introduction to Regular Expressions (Regex)

2.1 What are Regular Expressions?

Regular expressions are powerful patterns used to match, search, and manipulate text strings. They provide a standardized way to describe search patterns in text, making them an essential tool in programming, text processing, and data validation.

2.2.1 Pattern Matching

A regex pattern is a sequence of characters that defines a search pattern. These patterns can be:

- Literal characters that match themselves;
- Special characters (metacharacters) with special meanings;
- Combinations of both.

2.2.2 Basic Metacharacters

Metacharacter	Description	Example
`.`	Any character except newline	`a.c` matches “abc”, “a1c”, “a@c”
`^`	Matches start of string	`^Hello` matches “Hello World”
`$`	Matches end of string	`world$` matches “Hello world”
`*`	Matches 0 or more occurrences	`ab*c` matches “ac”, “abc”, “abbc”
`+`	Matches 1 or more occurrences	`ab+c` matches“abc”, “abbc”, “abbc”
`?`	Matches 0 or 1 occurrence	`ab?c` matches “ac” and “abc”
`\`	Escapes special characters	`\.` matches literal dot
`\w`	Matches word characters	a-z, A-Z, 0-9 and underscore

2.3 Common Use Cases

1. **Search Operations**
   - Advanced find/replace operations;
   - Pattern matching in large text files;
   - Content filtering.
2. **Text Processing**
   - Finding patterns in text;
   - Replacing specific text patterns;
   - Extracting information;
   - Parsing log files.
3. **Data Validation**
   - Email addresses;
   - Phone numbers;
   - Postal codes;
   - Passwords;
   - URLs.

2.4.1 Character Classes

```python
# Character class examples
pattern = r'[aeiou]'  # Matches any vowel
pattern = r'[0-9]'    # Matches any digit
pattern = r'[^0-9]'   # Matches any non-digit
```

2.4.2 Quantifiers and Groups

Python

```python
# Quantifiers
pattern = r'\d{3}'      # Exactly 3 digits
pattern = r'\d{2,4}'    # Between 2 and 4 digits
pattern = r'\d{2,}'     # 2 or more digits

# Groups
pattern = r'(\w+)\s+\1'  # Matches repeated words
```

2.4.3 Common Regex Functions in Python

Python

```python
import re

text = "The price is $19.99"

# Different matching functions
re.search(r'\$\d+\.\d+', text)   # Finds first match
re.findall(r'\$\d+\.\d+', text)  # Finds all matches
re.sub(r'\$(\d+\.\d+)', r'\1', text)  # Substitution

# Splitting text
re.split(r'\s+', text)  # Split on whitespace
```

2.5.1 Basic Pattern Matching

Python

```python
import re
# Simple pattern matching
text = "The quick brown fox jumps over the lazy dog"
pattern = r"fox"
# Search for pattern
match = re.search(pattern, text)
if match:
    print(f"Found '{pattern}' at position: {match.start()}-{match.end()}")
# Find all occurrences
words = re.findall(r"\w+", text)
print(f"All words: {words}")
```

2.5.2 Email Validation Example

Python

```python
def is_valid_email(email):
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(pattern, email))
# Test cases
emails = [
    "user@example.com", # ✓
    "invalid.email@com", # ✗
    "user.name@bizerte.r-iset.tn", # ✓
    "@invalid.com" # ✗
]
for email in emails:
    print(f"{email} → {'Valid' if is_valid_email(email) else 'Invalid'}")
```

2.5.3 Phone Number Formatting

Python

```python
def format_phone_number(phone):
    # Remove all non-digit characters
    digits = re.sub(r'\D', '', phone)

    # Format as (XXX) XXX-XXXX
    if len(digits) == 10:
        pattern = r'(\d{3})(\d{3})(\d{4})'
        formatted = re.sub(pattern, r'(\1) \2-\3', digits)
        return formatted

    return "Invalid phone number"
```

2.5 Python Implementation

Python

```python
# Test cases
numbers = [
    "1234567890",
    "123-456-7890",
    "(123) 456-7890",
    "12345"
]

for number in numbers:
    print(f"{number} → {format_phone_number(number)}")
```

2.6 Character Shorthands & Non-Capturing Groups

Built-in shorthands match common character classes

- **\s** — any whitespace character (space, tab, newline…)

```
\s+
```

Matches the gap in "hello world"

- **\S** — any NON-whitespace character

```
\S+
```

Matches "hello" and "world" separately

2.6 Character Shorthands & Non-Capturing Groups

\W **\W** — any NON-word character (opposite of \w)

```
\W+
```

Matches "!@#" in "hello!@#world"

\b **\b** — word boundary (zero-width, between \w and \W)

```
\bcat\b
```

Matches "cat" in "the cat sat" — not in "concatenate"

(?:) **(?: ...)** — non-capturing group: group without storing a backreference

```
(?:ha)+
```

Matches "hahaha" as one unit — no capture group created

2.6 Character Shorthands & Non-Capturing Groups

Token	Name	Matches
`\s`	Whitespace	space · tab · newline · form-feed · carriage-return
`\S`	Non-whitespace	anything `\s` does not match
`\W`	Non-word char	anything `\w` does not match — punctuation, spaces, symbols
`\b`	Word boundary	zero-width position between word and non-word chars
`(?:)`	Non-capturing group	groups a sub-pattern without a capture slot

2.7.1 Lookahead — Positive & Negative

Lookahead checks what **follows** the current position — without consuming any characters (zero-width assertion).

POSITIVE **(?=...)** — match only if followed by

```
\d+(?= dollars)
```

Matches 100 in "100 dollars" — not in "100 euros"

NEGATIVE **(?!...)** — match only if NOT followed by

```
\d+(?! dollars)
```

Matches 100 in "100 euros" — not in "100 dollars"

2.7.2 Lookbehind — Positive & Negative

Lookbehind checks what **precedes** the current position — also zero-width. Most engines require fixed-length lookbehind patterns.

POSITIVE **(?<=...)** — match only if preceded by

```
(?⇐\$)\d+
```

Matches 500 in "$500" — not in "€500"

NEGATIVE **(?<!...)** — match only if NOT preceded by

```
(?<!\$)\d+
```

Matches 500 in "€500" — not in "$500"

2.7 Lookahead and lookbehind

Type	Syntax	Meaning
Positive Lookahead	`(?= … )`	followed by
Negative Lookahead	`(?! … )`	NOT followed by
Positive Lookbehind	`(?⇐ … )`	preceded by
Negative Lookbehind	`(?<! … )`	NOT preceded by

2.8 Best Practices

1. **Use Raw Strings**
   - Always prefix regex patterns with r to avoid escape character issues

```python
pattern = r'\d+'  # Better than '\d+'
```

2. **Compile Frequently Used Patterns**

```python
email_pattern = re.compile(r'^[\w\.-]+@[\w\.-]+\.\w+$')
# Use multiple times
email_pattern.match(email1)
email_pattern.match(email2)
```

3. **Be Specific**
   - Make patterns as specific as possible to avoid false matches;
   - Use start (^) and end ($) anchors when matching whole strings.
4. **Test Thoroughly**
   - Test with both valid and invalid inputs
   - Include edge cases in your tests

```python
def test_pattern(pattern, test_cases):
    regex = re.compile(pattern)
    for test, expected in test_cases:
        result = bool(regex.match(test))
        print(f"'{test}': {'✓' if result == expected else '✗'}")
```

Quote

Some people, when confronted with a problem, think "I know, I'll use regular expressions." Now they have two problems. - Jamie Zawinski

1. **Greedy vs. Non-Greedy Matching**

```python
# Greedy (default)
re.findall(r'<.*>', '<tag>text</tag>')  # ['<tag>text</tag>']

# Non-greedy: Add (lazy) `?`
re.findall(r'<.*?>', '<tag>text</tag>')  # ['<tag>', '</tag>']
```

2. **Performance Considerations**
   - Avoid excessive backtracking *(recursion)*;
   - Be careful with nested quantifiers;
   - Use more specific patterns when possible.

2.10 Applications

1. **Basic Pattern Matching**

```python
# Write a pattern to match dates in format DD/MM/YYYY
date_pattern = r'\d{2}/\d{2}/\d{4}'
```

2. **Data Extraction**

```python
# Extract all email addresses from text
text = "Contact us at support@example.com or sales@example.com"
emails = re.findall(r'[\w\.-]+@[\w\.-]+\.\w+', text)
```

3. **Password Validation**

```python
def is_strong_password(password):
    # At least 8 chars, 1 upper, 1 lower, 1 digit, 1 special
    pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$' # Positive Lookahead
    return bool(re.match(pattern, password))
```

2.10 Applications

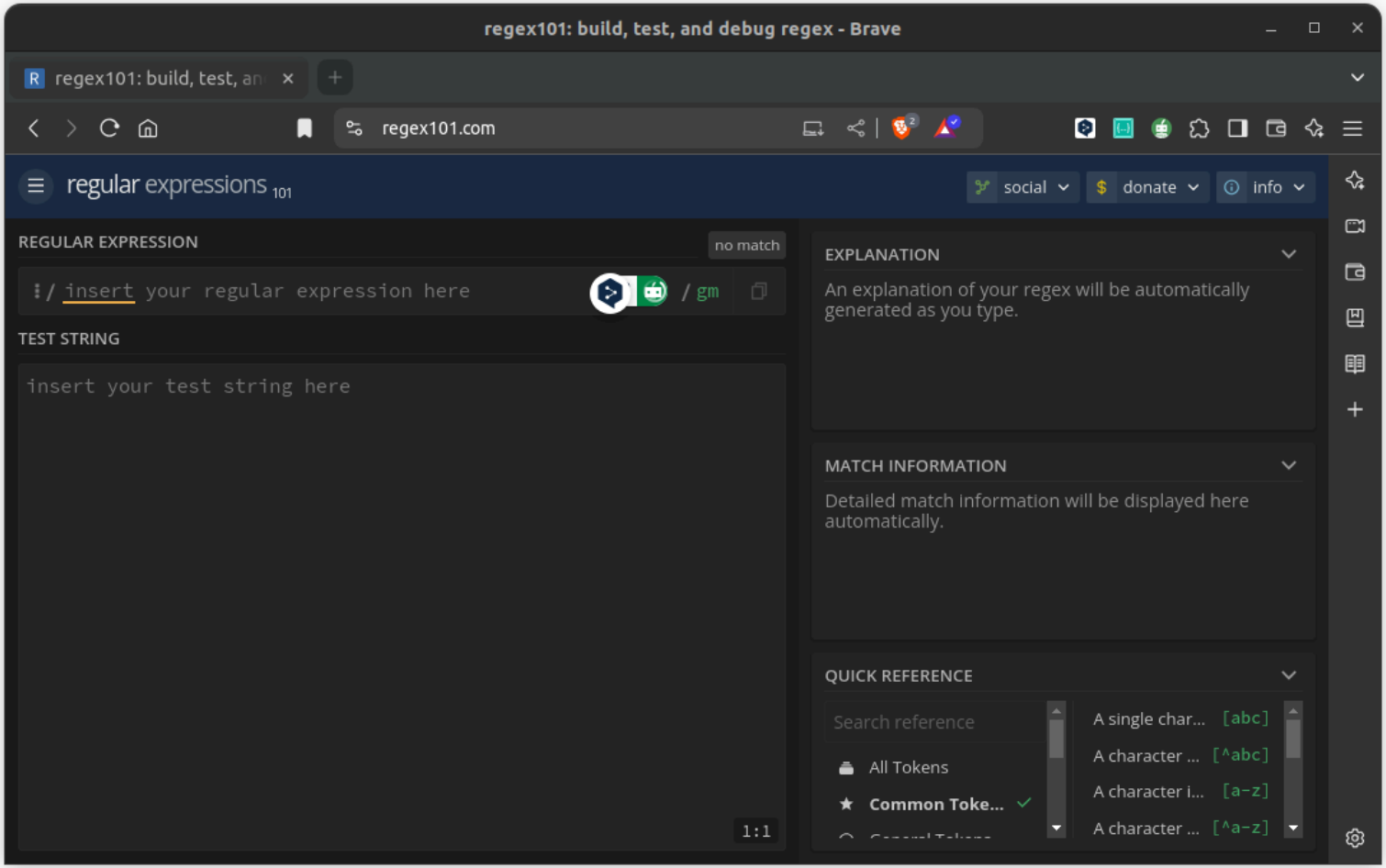


Figure 1: Build, test and debug regex patterns.

Task 1:

Write a regex pattern to match all occurrences of the word “python” (case-insensitive) in a string.

```
pattern = re.compile(r“python”, re.IGNORECASE)
```

Task 2:

Write a regex pattern to validate email addresses.

```
pattern = re.compile(r"^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+.[a-zA-Z]{2,}$")
```

Task 3:

Extract all phone numbers from a text where numbers can be in format XXX-XXX-XXXX or (XXX) XXX-XXXX.

```
pattern = re.compile(r"(\d{3}-\d{3}-\d{4}|\(\d{3}\)\s\d{3}-\d{4})")
```

Task 4:

Extract the username and domain from email addresses.

```
pattern = re.compile(r"^([a-zA-Z0-9._%+-]+)@([a-zA-Z0-9.-]+\.[a-zA-Z]{2,})$")
```

Task 5:

Match whole words “code” and “coding” but not words that contain them like “encoder” or “decode”.

```
pattern = re.compile(r“\b(code|coding)\b”)
```

3. Text Tokenization

3.1 Introduction to Tokenization

Tokenization is the process of breaking down text into smaller units called tokens. These tokens can be words, characters, subwords, or phrases depending on the specific requirements of the **NLP** task.

3.2.1 Simple Word Tokenization

Python

```python
import re

def simple_word_tokenize(text):
    # Split on whitespace and punctuation
    tokens = re.findall(r'\b\w+\b', text)
    return tokens

text = "Hello, world! This is a simple example."
tokens = simple_word_tokenize(text)
print(tokens)
```

['Hello', 'world', 'This', 'is', 'a', 'simple', 'example']

3.2.2 Advanced Regex Tokenization

Python

```python
def advanced_tokenize(text):
    pattern = r"""
        [a-zA-Z]+ | (?<\$)\d+(?:\.\d+)? | \d+(?:\.\d+)?(?=%)
    """
    tokens = re.findall(pattern, text)
    return tokens

text = "The price is $19.99, and the discount is 15%!"
print(advanced_tokenize(text))
```

['The', 'price', 'is', '19.99', 'and', 'the', 'discount', 'is', '15']

NLTK *(Natural Language Toolkit)* is a Python library offering tools for tokenization, stemming, part-of-speech tagging, and more, making it ideal for text analysis and linguistic research.

While it's widely used for education and prototyping, **NLTK** is less efficient for large-scale applications compared to modern libraries like **spaCy** or **Hugging Face Transformers**.

3.3.1 Installation and Usage

```python
import nltk
nltk.download('punkt_tab')  # Required for word and sentence tokenization
```

3.3.2 Word Tokenization

```python
from nltk.tokenize import word_tokenize, TreebankWordTokenizer
text = "Don't hesitate to use NLTK's tokenizer."
tokens = word_tokenize(text)
print(tokens)
# Output: ['Do', "n't", 'hesitate', 'to', 'use', 'NLTK', "'s", 'tokenizer', '.']
treebank = TreebankWordTokenizer()
tokens = treebank.tokenize(text)
print(tokens)
```

['Do', "n't", 'hesitate', 'to', 'use', 'NLTK', "'s", 'tokenizer', '.']

3.3.3 Sentence Tokenization

```python
from nltk.tokenize import sent_tokenize
text = """Mr. Smith bought a car. He loves driving it!
          What will he buy next? Only time will tell."""
sentences = sent_tokenize(text)
print(sentences)
```

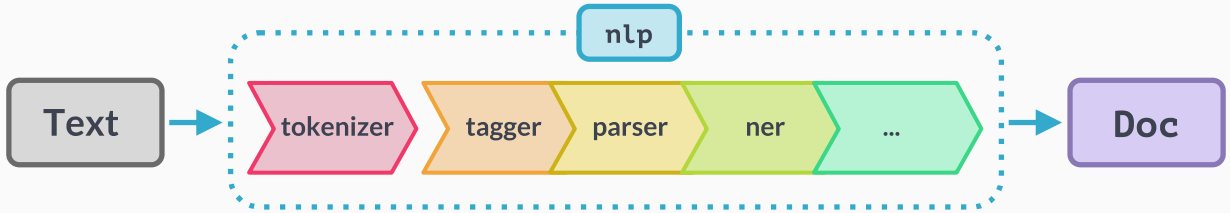
['Mr. Smith bought a car.', 'He loves driving it!', 'What will he buy next?', 'Only time will tell.']

3.3.4 Regular Expression Tokenizer

```python
from nltk.tokenize import RegexpTokenizer
tokenizer = RegexpTokenizer(pattern=r'\w+|[^\w\s]+')
text = "Hello, World! How's it going?"
tokens = tokenizer.tokenize(text)
print(tokens)
```

['Hello', ',', 'World', '!', 'How', '"', 's', 'it', 'going', '?']

spaCy[1,2] offers faster, more efficient tokenization with built-in linguistic annotations, making it better suited for production pipelines than rule-based alternatives.



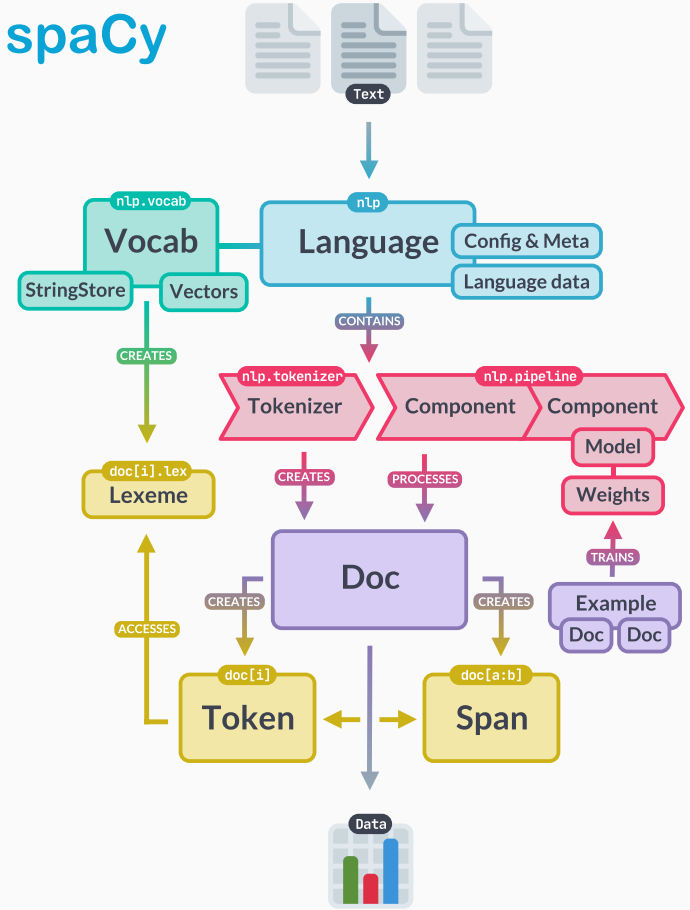
3.4.1 Installation and Usage

```bash
python -m spacy download en_core_web_sm
```

```python
import spacy
nlp = spacy.load('en_core_web_sm') # Load the English model
```

[1] https://spacy.io
[2] Free online course **Advanced NLP with spaCy** is available at https://course.spacy.io

3.4 spaCy Tokenization



3.4 spaCy Tokenization

```python
def spacy_tokenize(text):
    doc = nlp(text)
    return [token.text for token in doc]

text = "spaCy's tokenizer is industrial-strength!"
tokens = spacy_tokenize(text)
print(tokens)
```

[“spaCy”, “‘s“, “tokenizer”, “is”, “industrial”, “-”, “strength”, “!”]

3.4.2 Tokenization with Linguistic Features

Python

```python
text = "Apple's stock price rose 5.2% to $200 in 2024. Wow!"

# Tokenize and analyze the text
doc = nlp(text) # Transform the text into a Doc object
# Extract tokens with linguistic features
for token in doc:
    print(
        f"Token: {token.text:<10}",
        f"POS: {token.pos_:<8}",
        f"Lemma: {token.lemma_:<10}",
        f"Is punctuation? {token.is_punct}"
    )

```

3.4 spaCy Tokenization

```
"""
Token: Apple      POS: PROPN    Lemma: Apple      Is punctuation? False
Token: 's         POS: PART     Lemma: 's         Is punctuation? False
Token: stock      POS: NOUN     Lemma: stock      Is punctuation? False
Token: price      POS: NOUN     Lemma: price      Is punctuation? False
Token: rose       POS: VERB     Lemma: rise       Is punctuation? False
Token: 5.2        POS: NUM      Lemma: 5.2        Is punctuation? False
Token: %          POS: NOUN     Lemma: %          Is punctuation? True
Token: to         POS: ADP      Lemma: to         Is punctuation? False
Token: $          POS: SYM      Lemma: $          Is punctuation? False
Token: 200        POS: NUM      Lemma: 200        Is punctuation? False
 ...
"""
```

3.4 spaCy Tokenization

Python

```python
def analyze_tokens(text):
    """
    Analyze the tokens in a text using spaCy.
    """
    doc = nlp(text)
    for token in doc:
        print(f"""
        Text: {token.text}
        Lemma: {token.lemma_}
        POS: {token.pos_}
        Is stop word: {token.is_stop}
        """)

text = "Running quickly through the forest"
```

3.4 spaCy Tokenization

```
analyze_tokens(text)
"""
Text: Running
Lemma: run
POS: VERB
Is stop word: False

Text: quickly
Lemma: quickly
POS: ADV
Is stop word: False
"""
```

Polyglot is an open-source **NLP** library designed for multilingual text processing, supporting over 130 languages for tasks like tokenization, named entity recognition, and part-of-speech tagging. Unlike **spaCy** or **NLTK**, it specializes in low-resource languages, making it useful for linguistic diversity but less optimized for speed or deep learning integration.

3.5.1 Installation and Usage

```bash
pip install polyglot
```

```python
from polyglot.text import Text
```

3.5.2 Basic Usage

Python

```python
def polyglot_tokenize(text, lang=None):
    text = Text(text, hint_language_code=lang)
    return text.words

arabic_text = "مرحبا أيها العالم !"
english_text = "Hello, world!"
spanish_text = "¡Hola, mundo!"
samples = {"ar": arabic_text, "en": english_text, "es": spanish_text}

for k, v in samples:
    tokens = polyglot_tokenize(v, k)
    print(f"Original: {sample}")
    print(f"Tokens: {tokens}\n")
```

3.5 Polyglot Tokenization

```
"""
Original: ! مرحبا أيها العالم
Tokens: ['!', 'العالم' ,'أيها' ,'مرحبا']

Original: Hello, world!
Tokens: ['Hello', ',', 'world', '!']

Original: ¡Hola, mundo!
Tokens: ['¡', 'Hola', ',', 'mundo', '!']
"""
```

3.6.1 Regex vs. NLTK vs. spaCy vs. Polyglot

Python

```python
text = "Don't forget those in need!"

print("Regex:", simple_word_tokenize(text))
# Regex: ['Don', 't', 'forget', 'those', 'in', 'need']
print("NLTK:", word_tokenize(text))
# NLTK: ['Do', "n't", 'forget', 'those', 'in', 'need', '!']
print("spaCy:", spacy_tokenize(text))
# spaCy: ['Do', "n't", 'forget', 'those', 'in', 'need', '!']
print("Polyglot:", polyglot_tokenize(text))
# Polyglot: ["Don't", 'forget', 'those', 'in', 'need', '!']
```

3.6.2 Strengths and Use Cases

1. **Regex-based Tokenization**
   - 🛑 Simple, custom tokenization rules
   - 👍 Fast, flexible, easy to modify
   - 👎 Can't handle complex linguistic cases
2. **NLTK**
   - 🛑 Academic and research projects
   - 👍 Rich features, well-documented
   - 👎 Slower than some alternatives

3.6 Comparison of Different Tokenizers

3. **spaCy**
   - 🛑 Production environments
   - 👍 Fast, modern, good defaults
   - 👎 Larger memory footprint

4. **Polyglot**
   - 🛑 Multilingual projects
   - 👍 Excellent language coverage
   - 👎 Can be slower, fewer features

1. **Choose the Right Tokenizer**

```python
def select_tokenizer(text, language='en', needs_speed=False):
    if needs_speed and language == 'en':
        return spacy_tokenize(text)
    elif language ≠ 'en':
        return polyglot_tokenize(text)
    else:
        return word_tokenize(text)
```

3.7 Best Practices

Python

```python
# Sample texts for testing different tokenization paths
test_texts = {
    "english_basic": """This sentence contains simple punctuation, numbers (123), and some special characters !@#$.""",

    "english_complex": """Mr. Smith's car broke down at 3:30 P.M. "This is terrible," he thought.""",

    "multilingual": """Hello in French is Bonjour. In Spanish, hello is ¡Hola!""",

    "technical": """Python3 supports utf-8 encoding. Variables use snake_case by convention."""
}
```

Test case 1: Basic English, no speed requirement

```python
print("Test 1: Basic English (default settings)")
result1 = select_tokenizer(test_texts["english_basic"])
print(f"Tokens: {result1}\n")
```

['This', 'sentence', 'contains', 'simple', 'punctuation', ',', 'numbers', '(', '123', ')', ',', 'and', 'some', 'special', 'characters', '!', '@', '#', '$', '.']

Test case 2: Non-English text

```python
print("Test 2: Multilingual text")
result2 = select_tokenizer(test_texts["multilingual"], language="fr")
print(f"Tokens: {result2}\n")
```

['Hello', 'in', 'French', 'is', 'Bonjour', '.', 'In', 'Spanish', ',', 'hello', 'is', '¡', 'Hola', '!']

Test case 3: English with speed requirement

```python
print("Test 3: English with speed optimization")
result3 = select_tokenizer(test_texts["english_complex"], needs_speed=True)
print(f"Tokens: {result3}\n")
```

['Mr.', 'Smith', "'s", 'car', 'broke', 'down', 'at', '3:30', 'P.M.', '"', 'This', 'is', 'terrible', ',', '"', 'he', 'thought', '.']

Test case 4: Technical text with special characters

```python
print("Test 4: Technical text")
result4 = select_tokenizer(test_texts["technical"])
print(f"Tokens: {result4}")
```

['Python3', 'supports', 'utf-8', 'encoding', '.', 'Variables', 'use', 'snake_case', 'by', 'convention', '.']

2. **Pre-processing**

```python
def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    # Remove special characters (if needed)
    text = re.sub(r'[^\w\s]', '', text)
    return text
```

‘ Hello WORLD ‘

‘!@#$%^&*()_+’

‘Hello\n\tWorld\n Python’

‘hello world’

‘ _ ’

‘hello world python’

3. **Handling Special Cases**

```python
def handle_special_cases(tokens):
    """Expand contractions in a token list."""
    contractions = {"n't": "not", "'s": "is", "'re": "are"}  # Added common cases
    expanded_tokens = []
    for token in tokens:
        expanded_tokens.append(contractions.get(token, token)) # dict.get(key, default)
    return expanded_tokens
```

['They', 'ca', "n't", 'believe', 'it', "'s", 'already', 'Friday']

['They', 'ca', 'not', 'believe', 'it', 'is', 'already', 'Friday']

3.8.1 What is Named Entity Recognition?

Named Entity Recognition (NER) is a natural language processing technique that identifies and classifies named entities (key elements) in text into predefined categories.

The categories include but are not limited to:

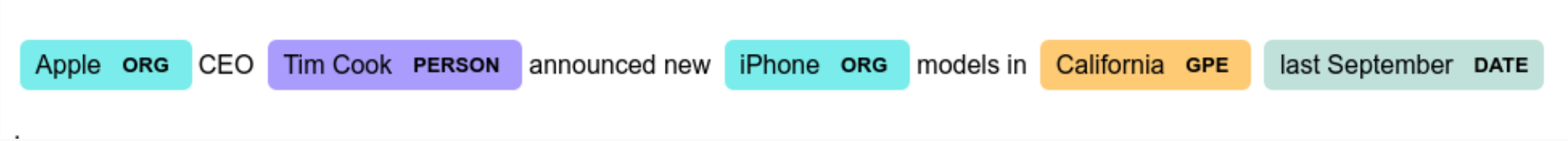
- Person names (e.g., “Barack Obama”, “Shakespeare”)
- Organizations (e.g., “Microsoft”, “United Nations”)
- Locations (e.g., “Paris”, “Mount Everest”)
- Date/Time expressions (e.g., “June 2024”, “last Monday”)
- Monetary values (e.g., “$1000”, “€50”)
- Percentages (e.g., “25%”, “three-quarters”)

3.8.2 Usage Example

Input text: *"Apple CEO Tim Cook announced new iPhone models in California last September."*

Identified entities:

- Apple (ORG)
- Tim Cook (PERSON)
- iPhone (ORG)
- California (GPE)
- September (DATE)



3.8 Named Entity Recognition (NER)

```python
import spacy
from spacy import displacy

nlp = spacy.load('en_core_web_sm')

doc = nlp("Apple CEO Tim Cook announced new iPhone models in California last September.")

displacy.serve(doc, style='ent')
```

Extracting Named Entities

Codes/extracting_named_entity.py

3.8.3 Common Use Cases

1. **Information Extraction**
   - Extracting company names from news articles
   - Identifying people mentioned in social media posts
   - Finding locations in travel blogs
2. **Document Classification**
   - Categorizing documents based on mentioned organizations
   - Sorting news articles by location
   - Grouping documents by date mentions

3. **Relationship Extraction**
   - Identifying business relationships between companies
   - Finding connections between people
   - Mapping event locations and dates
4. **Content Enrichment**
   - Adding metadata to documents
   - Linking entities to knowledge bases
   - Creating document summaries

4. Text Processing and Visualization

4.1.1 What is Text Preprocessing?

Text preprocessing is the process of cleaning and transforming raw text into a format that's more suitable for analysis.

Think of it as preparing ingredients before cooking - just as you wash and chop vegetables before cooking, you clean and standardize text before analysis.

1. Raw text: *"RT @username: Check out our new product!!! It's AMAZING... 😊 www.example.com #awesome"*
2. Preprocessed text: *"check out our new product it is amazing"*

4.1.2 Key Preprocessing Steps

4.1.2.1 Tokenization

The process of breaking down text into individual units (tokens), typically words or subwords.

1. Sentence: *"I love natural language processing!"*
2. Tokens: *["I", "love", "natural", "language", "processing", "!"]*
1. Sentence: *"Bizerte City is beautiful"*
2. Tokens: *["Bizerte", "City", "is", "beautiful"]*

4.1.2.2 Stop Word Removal

Eliminating common words that typically don't carry significant meaning.

Common stop words in English: "the", "is", "at", "which", "on", etc.

1. Original: *"The cat is on the mat"*
2. After stop word removal: *"cat mat"*

4.1.2.3 Lemmatization

Reducing words to their base or dictionary form (lemma).

```
am, are, is → be
running, ran, runs → run
better, best → good
wolves → wolf
```

4.1.2.4 Stemming

Reducing words to their root form by removing affixes, often resulting in non-dictionary words.

```
running → run
fishing → fish
completely → complet
authentication → authent
```

Let's analyze a customer review:

Original review:

"I've been using this phone for 3 months now... It's AMAZING!!! The battery life is incredible, and the camera takes beautiful pics. Can't believe how good it is :) Would definitely recommend to my friends & family!!!"

4.1 Fundamental Concepts and Definitions

1. **Cleaning**:
   "i have been using this phone for three months now it is amazing the battery life is incredible and the camera takes beautiful pictures cannot believe how good it is would definitely recommend to my friends and family"
2. **Tokenization**:
   ["i", "have", "been", "using", "this", "phone", "for", "three", "months", ...]
3. **Stop Word Removal**:
   ["phone", "three", "months", "amazing", "battery", "life", "incredible", "camera", "takes", "beautiful", "pictures", "good", "definitely", "recommend", "friends", "family"]
4. **Lemmatization**:
   ["phone", "month", "amazing", "battery", "life", "incredible", "camera", "take", "beautiful", "picture", "good", "definitely", "recommend", "friend", "family"]

4.1.3 Bag of Words (BoW)

A text representation method that describes the occurrence of words within a document. It creates a vocabulary of unique words and represents each document as a vector of word frequencies.

Documents:

1. "The cat likes milk"
2. "The dog hates milk"

Vocabulary:

["the", "cat", "dog", "likes", "hates", "milk"]

BoW representations:

- Doc 1: [1, 1, 0, 1, 0, 1]
- Doc 2: [1, 0, 1, 0, 1, 1]

- **Document 1**: "I love machine learning"
- **Document 2**: "I love deep learning"
- **Document 3**: "Deep learning is a subset of machine learning"

4.1.3.1 Vocabulary

The unique words across all documents: "I", "love", "machine", "learning", "deep", "is", "a", "subset", "of"

4.1.3.2 Document Vectors

Document 1: [1, 1, 1, 1, 0, 0, 0, 0, 0]

Document 2: [1, 1, 0, 1, 1, 0, 0, 0, 0]

Document 3: [0, 0, 1, 2, 1, 1, 1, 1, 1]

4.1.3.3 BoW Matrix

Document	I	love	machine	learning	deep	is	a	subset	of
Doc 1	1	1	1	1	0	0	0	0	0
Doc 2	1	1	0	1	1	0	0	0	0
Doc 3	0	0	1	2	1	1	1	1	1

4.1.3.4 Document Similarity Analysis

Let's verify document similarity by calculating dot products between document vectors:

$$\text{Doc 1} \cdot \text{Doc 2} = (1\times1) + (1\times1) + (1\times0) + (1\times1) + (0\times1) + (0\times0) + (0\times0) + (0\times0) + (0\times0) = 3$$

$$\text{Doc 1} \cdot \text{Doc 3} = (1\times0) + (1\times0) + (1\times1) + (1\times2) + (0\times1) + (0\times1) + (0\times1) + (0\times0) + (0\times1) = 3$$

$$\text{Doc 2} \cdot \text{Doc 3} = (1\times0) + (1\times0) + (0\times1) + (1\times2) + (1\times1) + (0\times1) + (0\times1) + (0\times1) + (0\times1) = 3$$

For proper similarity comparison, we should normalize using cosine similarity:

$$\cos(\theta) = \frac{(A) \cdot (B)}{\|(A)\| \times \|(B)\|}$$

Doc 1 magnitude: $\sqrt{1^2 + 1^2 + 1^2 + 1^2 + 0^2 + 0^2 + 0^2 + 0^2 + 0^2} = \sqrt{4} = 2$

Doc 2 magnitude: $\sqrt{1^2 + 1^2 + 0^2 + 1^2 + 1^2 + 0^2 + 0^2 + 0^2 + 0^2} = \sqrt{4} = 2$

Doc 3 magnitude: $\sqrt{0^2 + 0^2 + 1^2 + 2^2 + 1^2 + 1^2 + 1^2 + 1^2 + 1^2} = \sqrt{10} \approx 3.16$

Cosine similarities:

Doc 1 & Doc 2	**Doc 1 & Doc 3**	**Doc 2 & Doc 3**
$\frac{3}{2\times2} = 0.75$	$\frac{3}{2\times3.16} \approx 0.47$	$\frac{3}{2\times3.16} \approx 0.47$

Doc 1 and **Doc 2** are more similar to each other (0.75) than either is to **Doc 3** (0.47).

4.1.4 Term Frequency-Inverse Document Frequency (TF-IDF)

A numerical statistic that reflects how important a word is to a document in a collection of documents.

Consider these news articles:

1. "The new iPhone features advanced AI capabilities"
2. "The new Android phone launches today"
3. "The weather is nice today"

The word "the" appears in all documents, so it gets a low IDF score.
The word "iPhone" appears in only one document, so it gets a high IDF score.

4.1.4.1 TF-IDF Calculation

It is calculated by multiplying two metrics: Term Frequency (TF) and Inverse Document Frequency (IDF).

The TF-IDF score for a term t in document d from a document collection D is calculated as:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

1. **Term Frequency (TF)** measures how frequently term t appears in document d:

$$\mathrm{TF}(t, d) = \frac{\text{Number of times } t \text{ appears in } d}{\text{Total number of terms in } d}$$

2. **Inverse Document Frequency (IDF)** measures the importance of term t across all documents:

$$\mathrm{IDF}(t, D) = \log\left(\frac{N}{\mathrm{DF}(t)}\right)$$

Where:

- N is the total number of documents in collection D
- $\mathrm{DF}(t)$ is the number of documents containing term t

4.1.4.2 Computing Example

- **Document 1:** "This smartphone has a great camera and long battery life"
- **Document 2:** "The camera on this laptop is decent but the keyboard is excellent"
- **Document 3:** "Battery life is crucial for any smartphone"
- **Document 4:** "This laptop has a fast processor and excellent keyboard"

Calculate TF for each term in each document: First, let's count terms in each document (after removing common words like "this", "has", "a", "is", "the", etc.):

For **Document 1** (6 terms):

- $\text{TF}(\text{smartphone}) = \frac{1}{6} \approx 0.167$
- $\text{TF}(\text{great}) = \frac{1}{6} \approx 0.167$
- $\text{TF}(\text{camera}) = \frac{1}{6} \approx 0.167$
- $\text{TF}(\text{long}) = \frac{1}{6} \approx 0.167$
- $\text{TF}(\text{battery}) = \frac{1}{6} \approx 0.167$
- $\text{TF}(\text{life}) = \frac{1}{6} \approx 0.167$

For **Document 2** (5 terms):

- $\text{TF}(\text{camera}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{laptop}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{decent}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{keyboard}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{excellent}) = \frac{1}{5} = 0.2$

For **Document 3** (4 terms):

- $\text{TF}(\text{battery}) = \frac{1}{4} = 0.25$
- $\text{TF}(\text{life}) = \frac{1}{4} = 0.25$
- $\text{TF}(\text{crucial}) = \frac{1}{4} = 0.25$
- $\text{TF}(\text{smartphone}) = \frac{1}{4} = 0.25$

For **Document 4** (5 terms):

- $\text{TF}(\text{laptop}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{fast}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{processor}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{excellent}) = \frac{1}{5} = 0.2$
- $\text{TF}(\text{keyboard}) = \frac{1}{5} = 0.2$

Calculate IDF for each term across all documents:

Total documents (N) = 4

- $\text{IDF}(\text{smartphone}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{great}) = \log\left(\frac{4}{1}\right) \approx 0.602$
- $\text{IDF}(\text{camera}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{long}) = \log\left(\frac{4}{1}\right) \approx 0.602$
- $\text{IDF}(\text{battery}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{life}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{laptop}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{decent}) = \log\left(\frac{4}{1}\right) \approx 0.602$
- $\text{IDF}(\text{keyboard}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{excellent}) = \log\left(\frac{4}{2}\right) \approx 0.301$
- $\text{IDF}(\text{crucial}) = \log\left(\frac{4}{1}\right) \approx 0.602$
- $\text{IDF}(\text{fast}) = \log\left(\frac{4}{1}\right) \approx 0.602$
- $\text{IDF}(\text{processor}) = \log\left(\frac{4}{1}\right) \approx 0.602$

Calculate TF-IDF for each term in each document:

For **Document 1**:

- TF-IDF(smartphone, d_1) ≈ 0.05
- TF-IDF(great, d_1) ≈ 0.1
- TF-IDF(camera, d_1) ≈ 0.05
- TF-IDF(long, d_1) ≈ 0.1
- TF-IDF(battery, d_1) ≈ 0.05
- TF-IDF(life, d_1) ≈ 0.05

For **Document 2**:

- TF-IDF(camera, d_2) ≈ 0.06
- TF-IDF(laptop, d_2) ≈ 0.06
- TF-IDF(decent, d_2) ≈ 0.12
- TF-IDF(keyboard, d_2) ≈ 0.06
- TF-IDF(excellent, d_2) ≈ 0.06

For **Document 3**:

- TF-IDF(battery, d_3) ≈ 0.1
- TF-IDF(life, d_3) ≈ 0.1
- TF-IDF(crucial, d_3) ≈ 0.2
- TF-IDF(smartphone, d_3) ≈ 0.1

For **Document 4**:

- TF-IDF(laptop, d_4) ≈ 0.06
- TF-IDF(fast, d_4) ≈ 0.12
- TF-IDF(processor, d_4) ≈ 0.12
- TF-IDF(excellent, d_4) ≈ 0.06
- TF-IDF(keyboard, d_4) ≈ 0.06

The terms with the highest TF-IDF scores in each document:

Document	*Highest TF-IDF Terms*
Document 1	"great" (0.1) and "long" (0.1)
Document 2	"decent" (0.12)
Document 3	"crucial" (0.2)
Document 4	"fast" (0.12) and "processor" (0.12)

This demonstrates how TF-IDF identifies the most distinctive terms in each document. Terms that appear in only one document (like "crucial", "processor", or "fast") receive higher scores than terms that appear across multiple documents (like "smartphone" or "camera").

4.2 Understanding N-grams

An n-gram is a contiguous sequence of n items extracted from a text or speech sample. These items can be characters, words, syllables, or other linguistic units depending on the application context.

Unigrams (1-grams) Individual items *(e.g., single words like "computer")*
Bigrams (2-grams) Pairs of consecutive items *(e.g., "natural language")*
Trigrams (3-grams) Three consecutive items *(e.g., "machine learning algorithm")*
4-grams, 5-grams etc. Longer sequences following the same pattern

N-gram models operate on the **Markov** assumption that the probability of a word depends primarily on the previous $n - 1$ words, creating a probabilistic language model. While larger n values capture more context, they also increase computational complexity and data sparsity challenges.

4.2 Understanding N-grams

N-grams serve as fundamental building blocks in various **NLP** applications:

- Text prediction and auto-completion systems
- Machine translation
- Speech recognition
- Information retrieval and search engines
- Text classification and sentiment analysis
- Spelling correction
- Computational biology (for analyzing **DNA** sequences)

Task 6:

"The cat sat on the mat."

1. Identify all unigrams and their frequencies
2. List all possible bigrams
3. List all possible trigrams
4. Calculate the probability of "sat" following "cat"
5. If we add the sentence "The cat ate the fish," calculate the probability of "the" being followed by "mat"

4.2 Understanding N-grams

1. Unigrams and frequencies

- "The": 1
- "cat": 1
- "sat": 1
- "on": 1
- "the": 1
- "mat": 1

2. All possible bigrams:

- "The cat"
- "cat sat"
- "sat on"
- "on the"
- "the mat"

3. All possible trigrams:

- "The cat sat"
- "cat sat on"
- "sat on the"
- "on the mat"

If treating "The" and "the" as case-insensitive, then "the" would have a frequency of 2.

4. Probability of "sat" following "cat":

$$P(\text{sat}|\text{cat}) = \frac{\text{Count}(\text{cat sat})}{\text{Count}(\text{cat})} = \frac{1}{1} = 1.0$$

5. With added sentence "The cat ate the fish":

Combined text: "The cat sat on the mat. The cat ate the fish."

Updated frequencies for "the": 2 (if case-insensitive, counting "The" and "the") Count of "the mat": 1

$$P(\text{mat}|\text{the}) = \frac{\text{Count}(\text{the mat})}{\text{Count}(\text{the})} = \frac{1}{2} = 0.5$$

Task 7:

Text sample: "The dog ran fast. The dog jumped high. A cat ran past the dog. The cat was quick."

1. Count all unigrams, bigrams, and trigrams in the text (treat the text as case-insensitive)
2. Calculate the following probabilities:
   - P(ran|dog): Probability that "ran" follows "dog"
   - P(cat|The): Probability that "cat" follows "The"
   - P(high|jumped): Probability that "high" follows "jumped"
   - Using the bigram model, identify the most likely word to follow "The dog"
   - Using the bigram model, calculate the probability of the sequence "The cat ran"

4.2 Understanding N-grams

1. N-gram Counts (treating text as case-insensitive)

Unigrams (1-grams):	Bigrams (2-grams):	Trigrams (3-grams):
• "the": 4 • "dog": 3 • "ran": 2 • "fast": 1 • "jumped": 1 • "high": 1 • "a": 1 • "cat": 2 • "past": 1 • "was": 1 • "quick": 1	• "the dog": 3 • "dog ran": 1 • "ran fast": 1 • "fast the": 1 • "dog jumped": 1 • "jumped high": 1 • "high a": 1 • "a cat": 1 • "cat ran": 1 • "ran past": 1 • "past the": 1 • "the cat": 1	• "the dog ran": 1 • "dog ran fast": 1 • "ran fast the": 1 • "fast the dog": 1 • "the dog jumped": 1 • "dog jumped high": 1 • "jumped high a": 1 • "high a cat": 1 • "a cat ran": 1 • "cat ran past": 1 • "ran past the": 1 • "past the dog": 1

- "cat was": 1
- "was quick": 1

- "the dog the": 1
- "the cat was": 1
- "cat was quick": 1

2. Probability Calculations

$$P(\text{ran}|\text{dog}) = \frac{\text{Count}(\text{dog ran})}{\text{Count}(\text{dog})} = \frac{1}{3} = 0.33$$

$$P(\text{cat}|\text{The}) = \frac{\text{Count}(\text{the cat})}{\text{Count}(\text{the})} = \frac{1}{4} = 0.25$$

$$P(\text{high}|\text{jumped}) = \frac{\text{Count}(\text{jumped high})}{\text{Count}(\text{jumped})} = \frac{1}{1} = 1.0$$

To determine most likely word to follow "The dog" (using bigram model), we check the counts of "the dog X" for all X:

- Count("the dog ran") = 1
- Count("the dog jumped") = 1
- Count("the dog the") = 1

So there's a tie between with equal probability of 0.333 each ($\frac{1}{3}$).

Probability of the sequence "The cat ran" Using the bigram model and chain rule:

$$P(\text{The cat ran}) = P(\text{the}) \times P(\text{cat}|\text{the}) \times P(\text{ran}|\text{cat}) = \left(\frac{4}{18}\right) \times \left(\frac{1}{4}\right) \times \left(\frac{1}{2}\right) = 0.028$$

Where:

- $P(\text{the}) = \frac{4}{18}$ (frequency of "the" out of all words)
- $P(\text{cat}|\text{the}) = \frac{1}{4}$ (frequency of "the cat" out of all bigrams starting with "the")
- $P(\text{ran}|\text{cat}) = \frac{1}{2}$ (frequency of "cat ran" out of all bigrams starting with "cat")

Danger

In n-gram language models, we estimate the probability of word sequences. However, natural sentences have boundaries, and a model must know:

1. Where a sentence begins
2. Where a sentence ends

Without explicit delimiters, the model cannot correctly estimate probabilities for:

- the first word
- the last word
- transitions involving sentence boundaries

Success

We introduce special tokens:

<s> start of sentence (SOS)

</s> end of sentence (EOS)

Bigram Language Model with Sentence Boundaries

Corpus

```
<s> I love NLP </s>
<s> I love machine learning </s>
<s> I enjoy NLP </s>
```

Vocabulary: `<s>`, `I`, `love`, `enjoy`, `NLP`, `machine`, `learning`, `</s>`

Total sentences = 3

Goal: Estimate bigram probabilities.

Observed Bigrams

Bigram	Count
<s> I	3
I love	2
I enjoy	1
love NLP	1
love machine	1
enjoy NLP	1
NLP </s>	2
machine learning	1
learning </s>	1

Word Counts (Denominators)

Word	Count
<s>	3
I	3
love	2
enjoy	1
NLP	2
machine	1
learning	1

Bigram Probabilities

4.2 Understanding N-grams

$$P(\text{I} \mid \text{<s>}) = \frac{3}{3} = \mathbf{1.0}$$

$$P(\text{love} \mid \text{I}) = \frac{2}{3} \approx \mathbf{0.67}$$

$$P(\text{enjoy} \mid \text{I}) = \frac{1}{3} \approx \mathbf{0.33}$$

$$P(\text{NLP} \mid \text{love}) = \frac{1}{2} = \mathbf{0.5}$$

$$P(\text{machine} \mid \text{love}) = \frac{1}{2} = \mathbf{0.5}$$

$$P(\text{</s>} \mid \text{NLP}) = \frac{2}{2} = \mathbf{1.0}$$

Example

Sentence Probability

Sentence `<s> I love NLP </s>`

$$
\begin{aligned}
P &= P(\text{I} \mid \text{<s>}) \times P(\text{love} \mid \text{I}) \times P(\text{NLP} \mid \text{love}) \times P(\text{</s>} \mid \text{NLP}) \\
&= 1 \times 0.67 \times 0.5 \times 1 \\
&\approx \mathbf{0.335}
\end{aligned}
$$

Info

Handling Unseen **N-grams** *(Bonus)*

Add-one (Laplace) smoothing Add 1 to all counts to ensure no zero probabilities

$$P(w_2|w_1) = \frac{\text{Count}(w_1, w_2) + 1}{\text{Count}(w_1) + V}$$

Where V is the vocabulary size (number of unique words)

Add-k smoothing Add a small k ($0 < k < 1$) to all counts

Good-Turing smoothing Probability estimation is based on frequency of rare events.

Backoff models If a higher-order **n-gram** isn't observed, fall back to a lower-order **n-gram**

Interpolation Combine probabilities from different order **n-grams** with weights

4.3 Complete Pipeline Example

Text Processing

Codes/text_processing.py

4.4.1 Frequency Distribution Plots

Charts showing how often different words appear in a text.

- Comparing vocabulary usage across different authors;
- Analyzing social media hashtag popularity over time;
- Studying language patterns in different genres of literature.

4.4.2 Word Clouds

A visual representation where word size corresponds to its frequency in the text.

- Analyzing customer reviews to identify common themes;
- Visualizing key topics in political speeches;
- Summarizing survey responses.

Text Visualization

Codes/text_visualization.py

4.5 Common Use Cases and Applications

1. **Sentiment Analysis**
   - Customer review processing
   - Social media monitoring
   - Brand reputation tracking
2. **Content Classification**
   - News article categorization
   - Spam detection
   - Document sorting
3. **Text Summarization**
   - News article summarization
   - Document abstract generation
   - Meeting notes condensation
4. **Keyword Extraction**
   - SEO optimization
   - Content tagging
   - Research paper indexing

4.6 Best Practices and Tips

1. **Choose the Right Tools**
   - Use `NLTK` for research and experimentation
   - Use `spaCy` for production environments
   - Use `scikit-learn` for machine learning integration
2. **Performance Optimization**
3. **Error Handling**
4. **Evaluation Metrics**

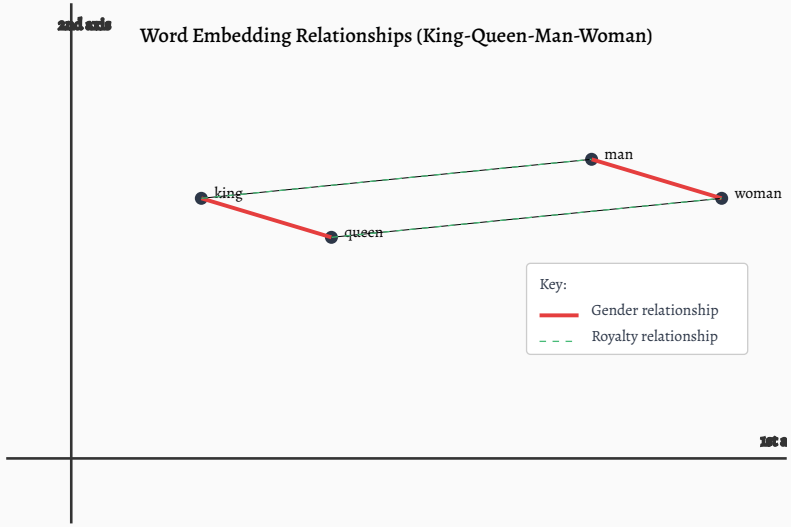
5. Word Embedding

Word embedding is a technique that represents words as dense vectors of real numbers in a continuous vector space. Words with similar meanings are placed close to each other in this space. It captures semantic relationships between words. For example, the embedding can learn that:

```
king - man + woman ≈ queen
```

Dimension Reduction Unlike sparse one-hot encodings, embeddings use lower-dimensional vectors *(e.g., 100-300 dimensions)*

Context Awareness Embeddings are learned from large text corpora, so they reflect word usage and context.

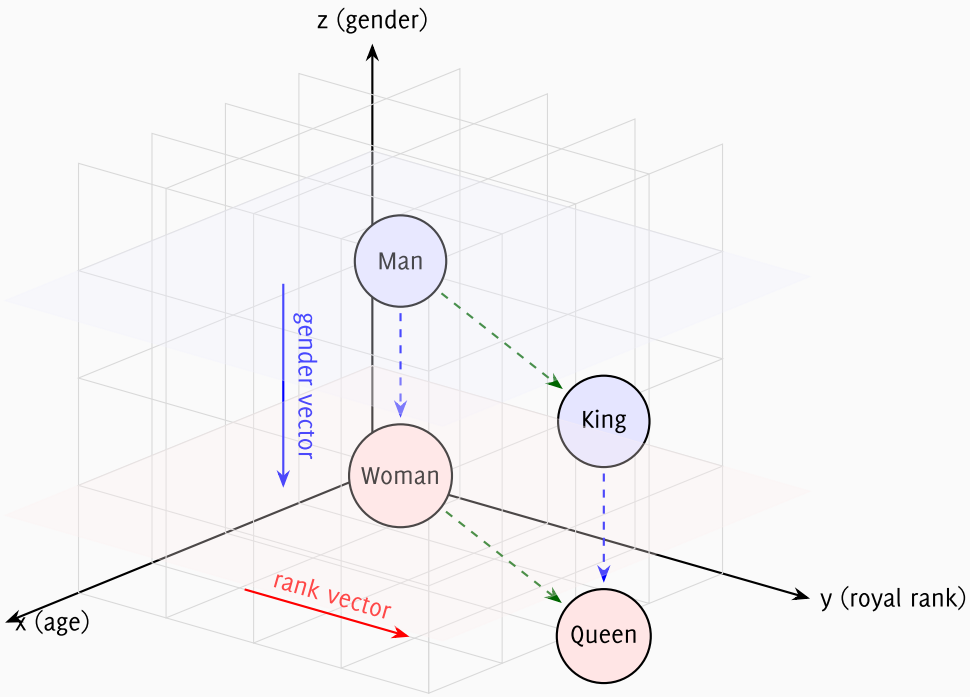


Given these words:

```
["king", "queen", "man", "woman"]
```

They might be embedded as:

```
king → [0.7, 0.1, ... ]
queen → [0.68, 0.11, ... ]
man → [0.6, -0.2, ... ]
woman → [0.58, -0.19, ... ]
```



These vectors preserve semantic relationships and can be used to perform reasoning, classification, translation, and even analogy tasks in a meaningful way. For example, if we want to find the word that is to "Italy" as "Paris" is to "France", we can use the embedding vectors:

```
["Paris", "France", "Rome", "Italy"]
```

In a well-trained word embedding space, the model learns not just word meanings but also analogical relationships. It can capture that:

```
Paris - France + Italy ≈ Rome
```

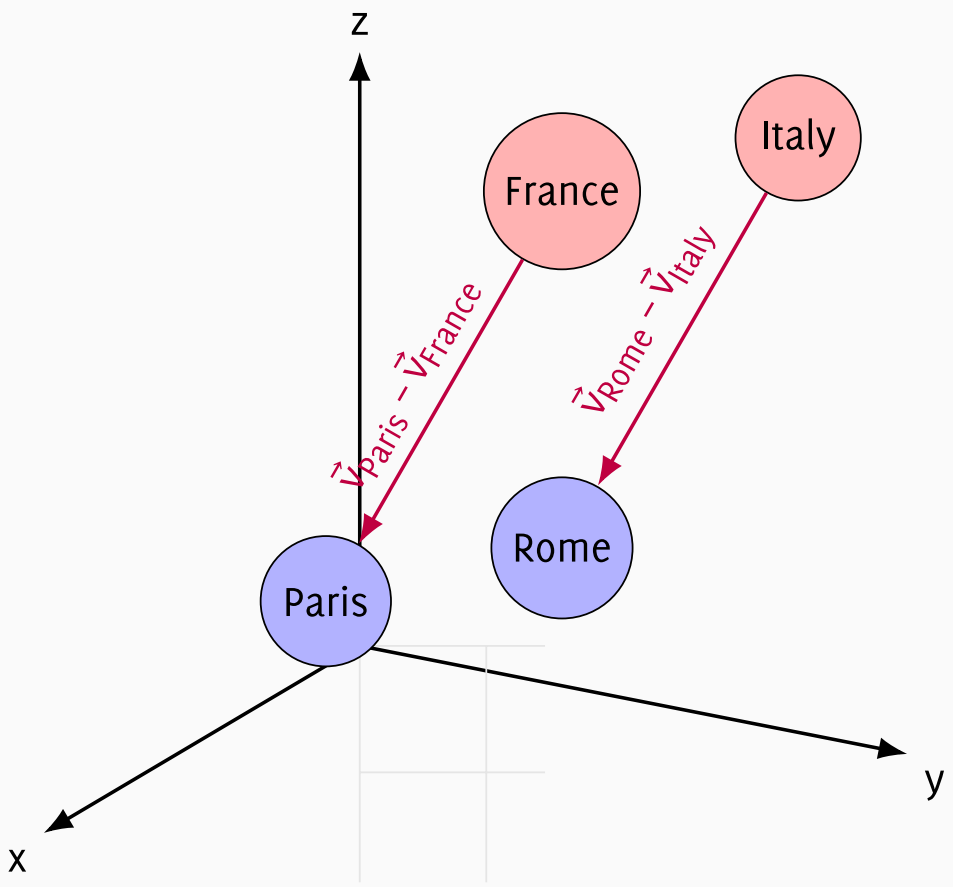
This means that the difference between "Paris" and "France" is similar to the difference between "Rome" and "Italy" — they are both capital-country pairs.

5. Word Embedding

```
Paris  → [0.5, 0.1, 0.6, ... ]
France → [0.4, 0.2, 0.5, ... ]
Rome → [0.52, 0.12, 0.62, ... ]
Italy → [0.42, 0.22, 0.52, ... ]
```

If we compute:

```
Paris - France + Italy
     ≈ [0.5-0.4+0.42,
        0.1-0.2+0.22,
        0.6-0.5+0.52]
     = [0.52, 0.12, 0.62]
     ≈ Rome
```



5.1 Types of Word Embedding Models

Models like `Word2Vec`, `GloVe`, and `FastText` are commonly used to generate these embeddings.

Word2Vec Introduced by Google; Predicts surrounding words given a target word (CBOW) or vice versa (Skip-gram).

GloVe (Global Vectors) Developed by Stanford; leverages global word co-occurrence statistics.

FastText Developed by Facebook; includes subword information to handle rare and morphologically complex words.

5.1.1 Word2Vec

Two variants

1. **Skip-Gram**: Predicts context words from a target word.
2. **CBOW (Continuous Bag-of-Words)**: Predicts target word from context.

- Sliding window captures local context relationships.
- Captures semantic/grammatical analogies (*e.g.*, `king - man + woman ≈ queen`).
- Lightweight and scalable for large datasets.
- Excels at contextual similarity tasks (*e.g.*, *word clustering*).
- Pre-trained embeddings (*e.g.*, *Google News, 300D*) for plug-and-play NLP.
- Foundational for advanced models (*RNNs, Transformers*).

5.1.2 GloVe (Global Vectors)

- Hybrid approach: Combines **global corpus co-occurrence statistics** with **local context patterns**.
- Weighted loss function reduces impact of rare/frequent word pairs.
- Precomputed co-occurrence matrix speeds up training.
- Captures linear relationships (*e.g.*, `king - man + woman ≈ queen`).
- Pre-trained embeddings (*e.g.*, *Wikipedia, Common Crawl, 50D-300D*).
- Outperforms `Word2Vec` on semantic similarity tasks.
- Scalable, interpretable, and efficient for large corpora.

5.1.3 FastText

- Extends `Word2Vec` by incorporating *subword information* (character n-grams).
- Represents words as a bag of character n-grams *(e.g., "apple" → "ap", "app", "pple")*.
- Uses **skip-gram** or **CBOW** with subword embeddings.
- Handles rare/out-of-vocabulary (**OOV**) words via subword components.
- Robust for morphologically rich languages *(e.g., Turkish, Finnish)*.
- Pre-trained vectors for 157+ languages *(e.g., Wikipedia, Common Crawl)*.
- Better at handling typos, slang, and rare words.
- Strong performance in text classification and low-resource languages.
- Beats **Word2Vec** in semantic tasks
- Scalable, interpretable, integrates into **NLP** pipelines

5.2.1 What is Gensim?

Gensim is a robust, efficient library for topic modeling, document indexing, and similarity retrieval with large corpora. The name **Gensim** stands for "Generate Similar" - reflecting its core functionality of finding similar documents.

Key features:

- Memory efficient processing of large text collections
- Built-in implementations of popular algorithms like **Word2Vec**, **Doc2Vec**, **FastText**
- Streamlined document similarity calculations
- Topic modeling capabilities (**LSA**, **LDA**)

5.2.2 Basic Gensim Usage

5.2.2.1 Installation and Usage

```bash
pip install gensim
```

```python
import gensim
from gensim import corpora, models
```

Building a Complete Text Analysis Pipeline

Codes/gensim_text_processing.py

5.2.3 Best Practices and Tips

1. **Preprocessing**
   - Convert to lowercase
   - Remove stop words
   - Apply lemmatization
   - Handle special characters

2. **Memory Efficiency**
   - Use streaming corpus for large datasets
   - Implement memory-efficient iterators

```python
class MyCorpus:
    def __iter__(self):
        for line in open('mycorpus.txt'):
            yield dictionary.doc2bow(line.lower().split())
```

3. **Model Persistence**

Save models

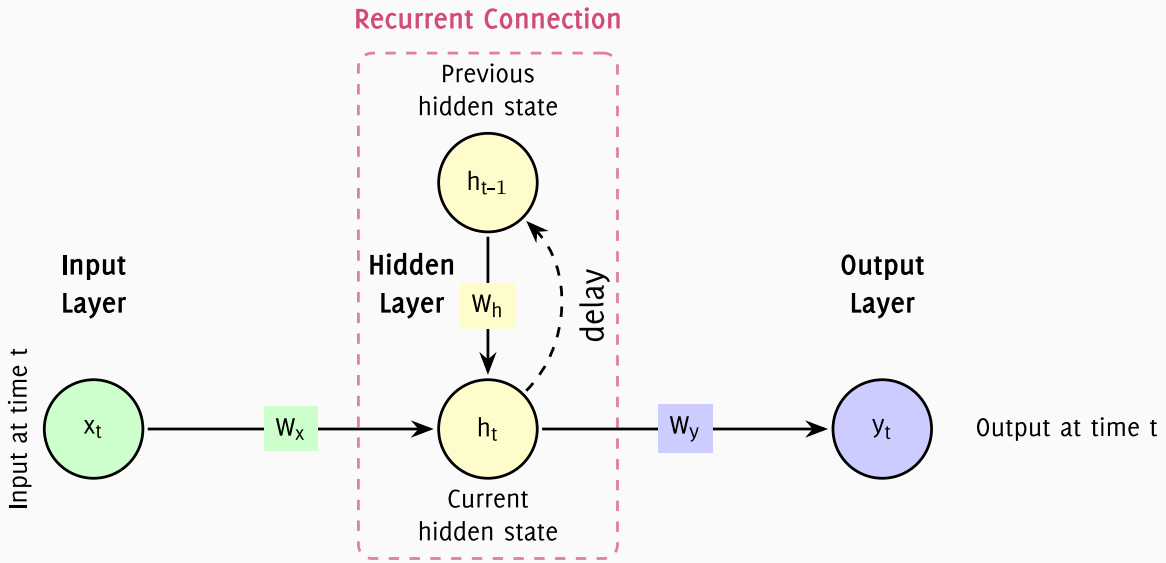
```python
dictionary.save('dictionary.gensim')
tfidf_model.save('tfidf.gensim')
```

Load models

```python
dictionary = corpora.Dictionary.load('dictionary.gensim')
tfidf_model = models.TfidfModel.load('tfidf.gensim')
```

6. Deep Learning for NLP

6.1 Recurrent Neural Network (RNN)



Legend:
- Input Nodes
- Hidden State
- Output Nodes

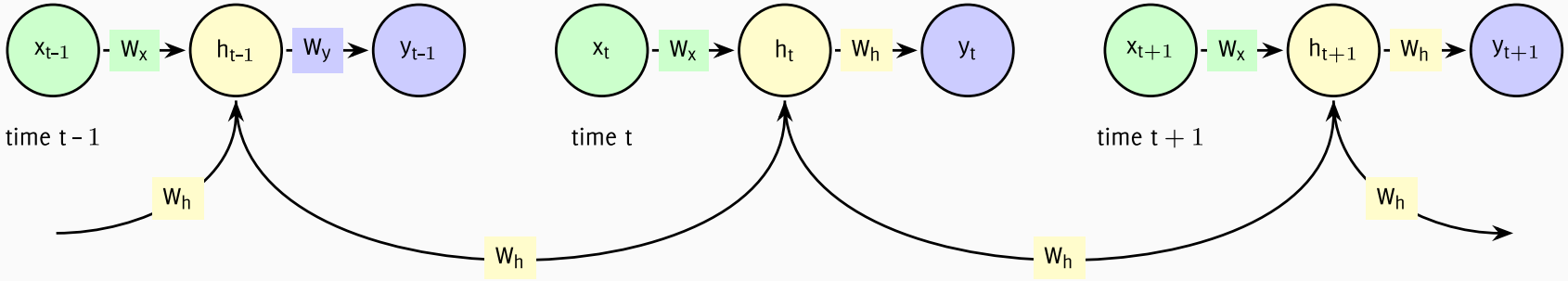
RNN Equations:

Hidden state: $h_t = f(W_x \cdot x_t + W_h \cdot h_{t-1} + b_h)$

Output: $y_t = f(W_y \cdot h_t + b_y)$

- *tanh or ReLU for hidden layer,*
- *softmax/sigmoid for output*

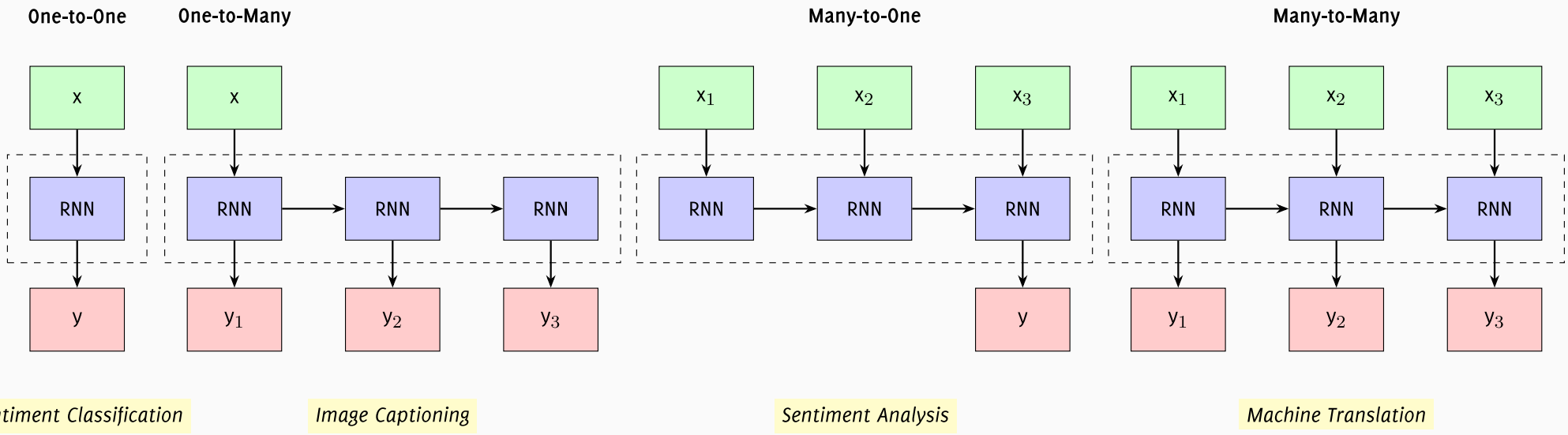
Unfolded RNN (Through Time)



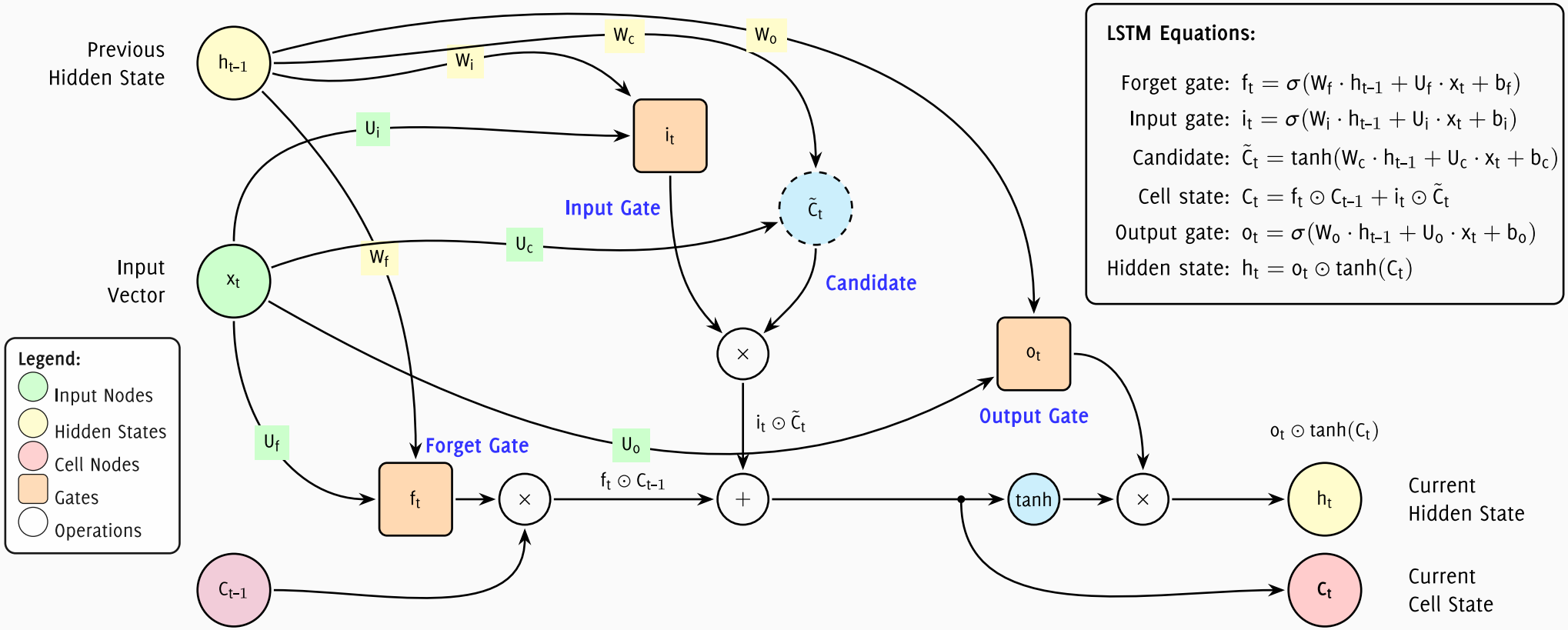
6.1 Recurrent Neural Network (RNN)

- Processes sequential data
- Shares parameters across time
- Memory of previous inputs
- Useful for text, speech, time series
- Struggles with long-term dependencies
- Prone to vanishing/exploding gradients
- Suitable for tasks like sentiment analysis, text classification, and sequence labeling.

One-to-One A standard **RNN** that takes a single input and produces a single output
One-to-Many Takes a single input and produces a sequence of outputs
Many-to-One Takes a sequence of inputs and produces a single output
Many-to-Many Takes a sequence of inputs and produces a sequence of outputs



6.2 Long Short-Term Memory (LSTM)



LSTM Equations:

Forget gate: $f_t = \sigma(W_f \cdot h_{t-1} + U_f \cdot x_t + b_f)$

Input gate: $i_t = \sigma(W_i \cdot h_{t-1} + U_i \cdot x_t + b_i)$

Candidate: $\tilde{C}_t = \tanh(W_c \cdot h_{t-1} + U_c \cdot x_t + b_c)$

Cell state: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$

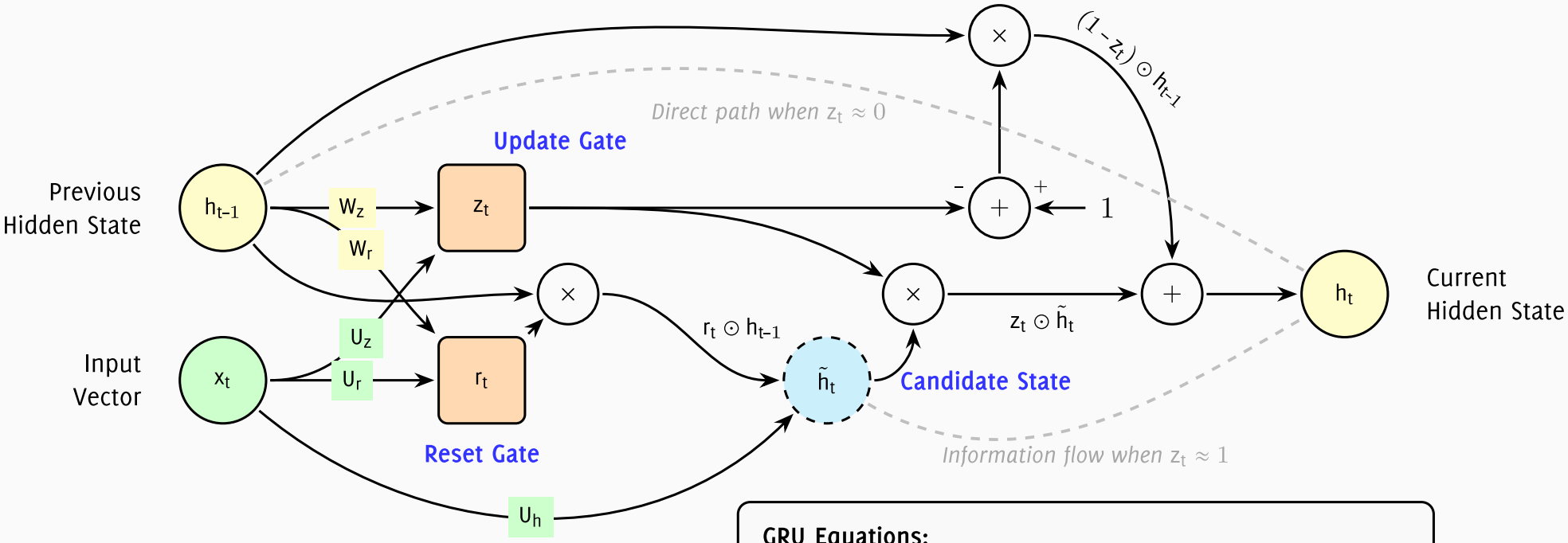
Output gate: $o_t = \sigma(W_o \cdot h_{t-1} + U_o \cdot x_t + b_o)$

Hidden state: $h_t = o_t \odot \tanh(C_t)$

6.2 Long Short-Term Memory (LSTM)

- Extends **RNN** by adding a cell state.
- Uses gates *(input, forget, output)* to control information flow.
- Better at capturing long-term dependencies.
- Uses a cell state to carry information across time steps.
- Can be slower to train due to the complexity of the architecture.
- Can be computationally expensive for long sequences.

6.3 Gated Recurrent Unit (GRU)



Legend:

- Input Nodes
- Hidden States
- Gates
- Operations

GRU Equations:

$$\text{Update Gate: } z_t = \sigma(W_z \cdot h_{t-1} + U_z \cdot x_t + b_z)$$

$$\text{Reset Gate: } r_t = \sigma(W_r \cdot h_{t-1} + U_r \cdot x_t + b_r)$$

$$\text{Candidate State: } \tilde{h}_t = \tanh(W_h \cdot (r_t \odot h_{t-1}) + U_h \cdot x_t + b_h)$$

$$\text{Hidden State: } h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

6.3 Gated Recurrent Unit (GRU)

- A simplified version of **LSTM**.
- Uses update gate (z_t) to control information flow
- Uses reset gate (r_t) to control the hidden state
- Combines input and forget gates into a single update gate.
- Less complex than **LSTM**, making it easier to implement and understand.
- Can be faster to train due to fewer parameters.

RNN vs LSTM vs GRU

- **RNNs** are suitable for simpler tasks with shorter sequences, but struggle with long-term dependencies.
- **LSTMs** are more complex but better at capturing long-term dependencies.
- **GRUs** are a middle ground, offering a simpler architecture with comparable performance to **LSTMs**.
- **LSTMs** and **GRUs** are often used in applications where long-term dependencies are crucial, such as language modeling and text generation.

Sequence Processing Foundations

Jupyter/sequence_processing.ipynb

6.4 Transformer

The **Transformer** is a neural network architecture that has become the foundation of most modern NLP models. Introduced in 2017 by Vaswani et al. in the paper **"Attention Is All You Need"**[1], it replaced earlier sequence models like RNNs and LSTMs by introducing a fully attention-based mechanism.

Transformers have significantly advanced the field of NLP by enabling models to learn deeper context, handle longer dependencies, and train faster due to parallel computation. Unlike RNNs, which process input sequentially, **Transformers** can process all tokens at once. This allows faster training and better scalability on large datasets.

[1] https://arxiv.org/abs/1706.03762

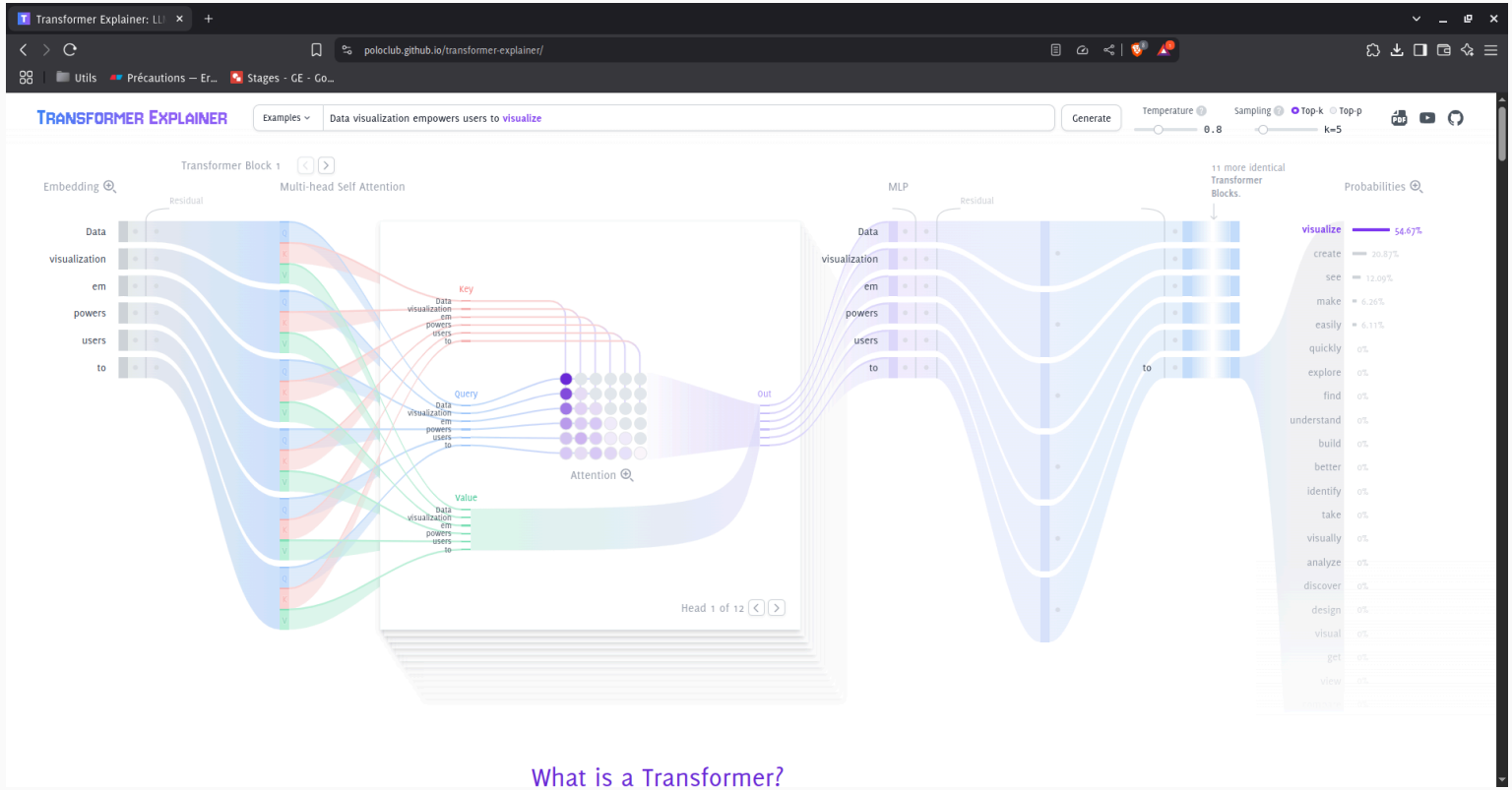


Figure 5: Transformer explained visually[1].

[1] https://poloclub.github.io/transformer-explainer/

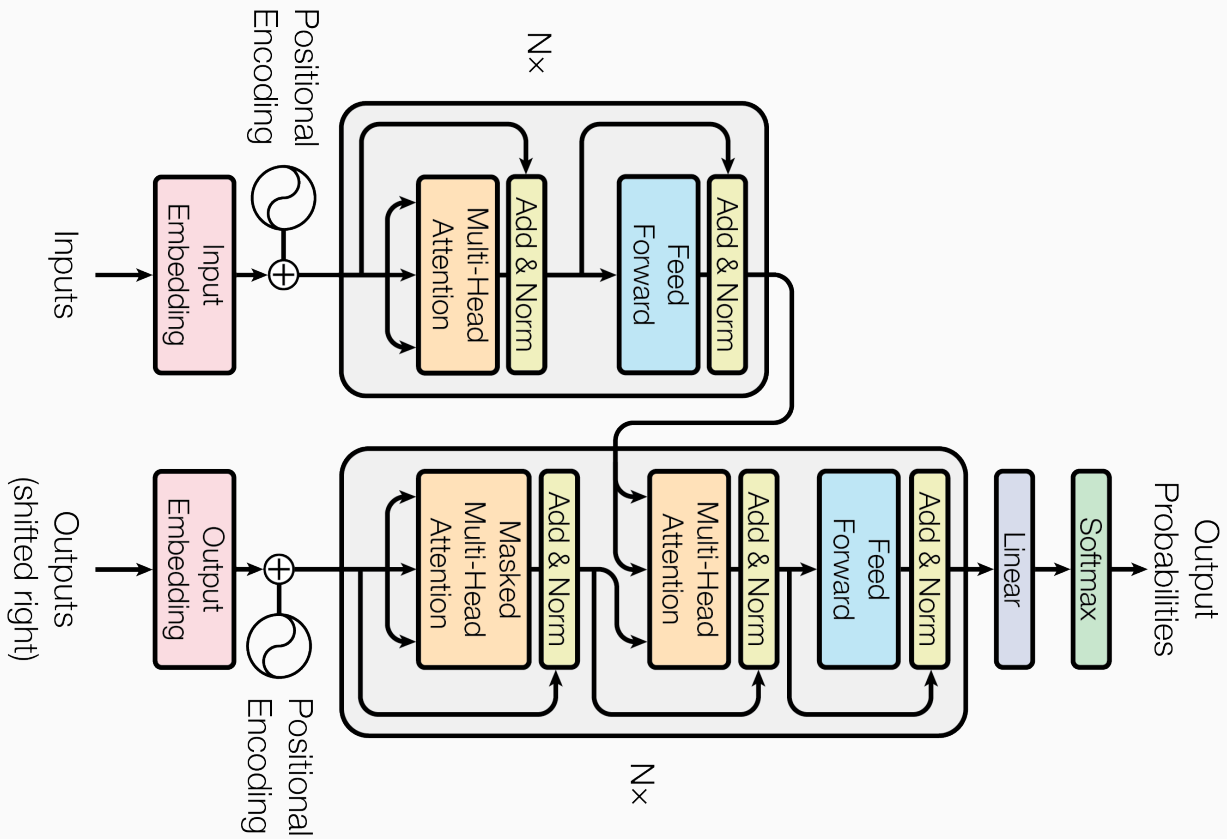


Figure 6: Core Components[1].

[1] https://arxiv.org/abs/1706.03762

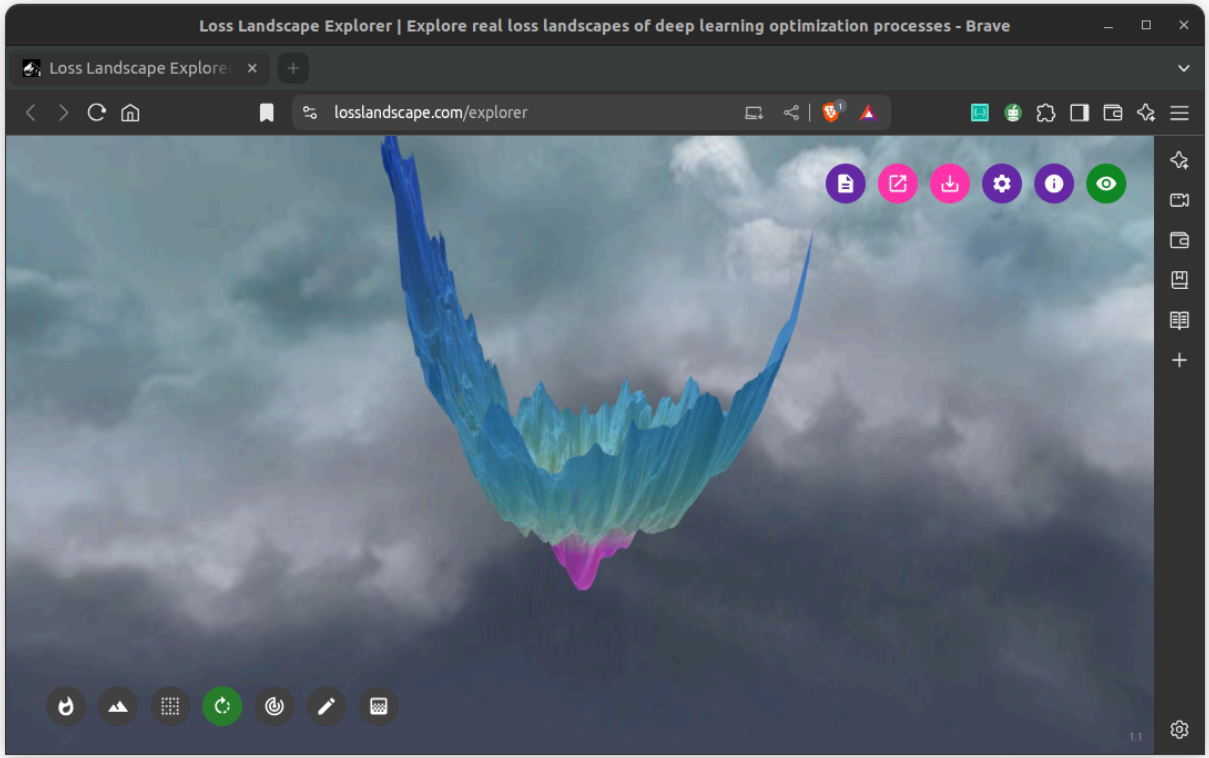


Figure 7: Why Skip Connection[1]?

[1] https://losslandscape.com/explorer

6.4.1 Positional Encoding

- Transformers don't inherently understand word order.
- Positional encodings are added to word embeddings to inject information about the position of words in a sentence.

Task 8:

We have a short sentence with `sequence_length = 4` words: `["I", "like", "to", "learn"]`.

The initial learned embeddings for these words are *(The embedding size is* `d_model = 5`*)*:

E("I") = `[0.1, 0.2, 0.1, -0.3, 0.5]`
E("like") = `[-0.3, 0.4, 0.6, -0.2, 0.1]`
E("to") = `[0.5, -0.1, 0.0, 0.3, 0.2]`
E("learn") = `[0.2, 0.6, -0.4, 0.1, 0.7]`

1. Calculate the positional encoding vector for the word "like", which is at position `pos = 1`.
2. Calculate the positional encoding vector for the word "learn", which is at position `pos = 3`.

$$\text{PE}\ (\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{\text{d_model}}}}\right) \text{ and } \text{PE}\ (\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{\frac{2i+1}{\text{d_model}}}}\right)$$

1. "like" at `pos = 1`:

$$i = 0 \Rightarrow \mathrm{PE}(1,0) = \sin\left(\frac{1}{10000^{\frac{0}{5}}}\right) \approx 0.8415$$

$$i = 1 \Rightarrow \mathrm{PE}(1,1) = \cos\left(\frac{1}{10000^{\frac{0}{5}}}\right) \approx 0.5403$$

$$i = 2 \Rightarrow \mathrm{PE}(1,2) = \sin\left(\frac{1}{10000^{\frac{2}{5}}}\right) \approx 0.0251$$

$$i = 3 \Rightarrow \mathrm{PE}(1,3) = \cos\left(\frac{1}{10000^{\frac{2}{5}}}\right) \approx 0.9997$$

$$i = 4 \Rightarrow \mathrm{PE}(1,4) = \sin\left(\frac{1}{10000^{\frac{4}{5}}}\right) \approx 0.0006$$

2. "learn" at `pos = 3`:

$$i = 0 \Rightarrow \mathrm{PE}(3,0) = \sin\left(\frac{3}{10000^{\frac{0}{5}}}\right) \approx 0.1411$$

$$i = 1 \Rightarrow \mathrm{PE}(3,1) = \cos\left(\frac{3}{10000^{\frac{0}{5}}}\right) \approx -0.9900$$

$$i = 2 \Rightarrow \mathrm{PE}(3,2) = \sin\left(\frac{3}{10000^{\frac{2}{5}}}\right) \approx 0.0753$$

$$i = 3 \Rightarrow \mathrm{PE}(3,3) = \cos\left(\frac{3}{10000^{\frac{2}{5}}}\right) \approx 0.9972$$

$$i = 4 \Rightarrow \mathrm{PE}(3,4) = \sin\left(\frac{3}{10000^{\frac{4}{5}}}\right) \approx 0.0019$$

Positional Encoding Calculations

Word	Position (pos)	PE Vector *(rounded to 4 decimals)*
"like"	1	[0.8415, 0.5403, 0.0251, 0.9997, 0.0006]
"learn"	3	[0.1411, −0.9900, 0.0753, 0.9972, 0.0019]

Final Input Vectors (Embedding + PE)

Word	Final Input Vector *(rounded to 4 decimals)*
"like"	[0.5415, 0.9403, 0.6251, 0.7997, 0.1006]
"learn"	[0.3411, −0.3900, −0.3247, 1.0972, 0.7019]

Python

```python
# Create a matrix of zeros to store positional encodings
pe = torch.zeros(max_seq_length, d_model)
# Generate positions and add a dimension for broadcasting
pos = torch.arange(0, max_seq_length, dtype=torch.float).unsqueeze(1)
# division term for the sine and cosine arguments (1 / 10000^(2i/d_model))
div_term = torch.exp(torch.arange(0, d_model, 2).float() * -(math.log(10000.0) / d_model))

# Apply sine to even indices in the embedding (2i)
pe[:, 0::2] = torch.sin(pos * div_term)
# Apply cosine to odd indices in the embedding (2i+1)
pe[:, 1::2] = torch.cos(pos * div_term)
```

6.4.2 Encoder-Decoder Structure

- **Encoder**: Takes the input sequence and encodes it into a context-aware representation.
- **Decoder**: Uses the encoded representation to generate output *(e.g., in translation or summarization)*.

6.4.3 Self-Attention

- Each word in a sentence pays attention to every other word to understand context.
- This allows the model to dynamically weigh the importance of different words.
- Example: In the sentence **"She poured water into the glass until it was full."**, the model can learn that **"it"** refers to **"glass"**.

> **Quote**
>
> *The key/value/query concept is analogous to retrieval systems. For example, when you search for videos on Youtube, the search engine will map your query (text in the search bar) against a set of keys (video title, description, etc.) associated with candidate videos in their database, then present you the best matched videos (values)*[1].

[1] https://stats.stackexchange.com/questions/421935/what-exactly-are-keys-queries-and-values-in-attention-mechanisms

6.4 Transformer

Architecture

- Multi-head self-attention layers
- Feed-forward neural networks
- Layer normalization
- Residual connections

Use Cases

- Machine Translation
- Text Generation
- Question Answering
- Text Classification
- Summarization

Popular Models

- `BERT`
- `GPT`
- `T5`
- `RoBERTa`, `XLNet`, `DistilBERT`

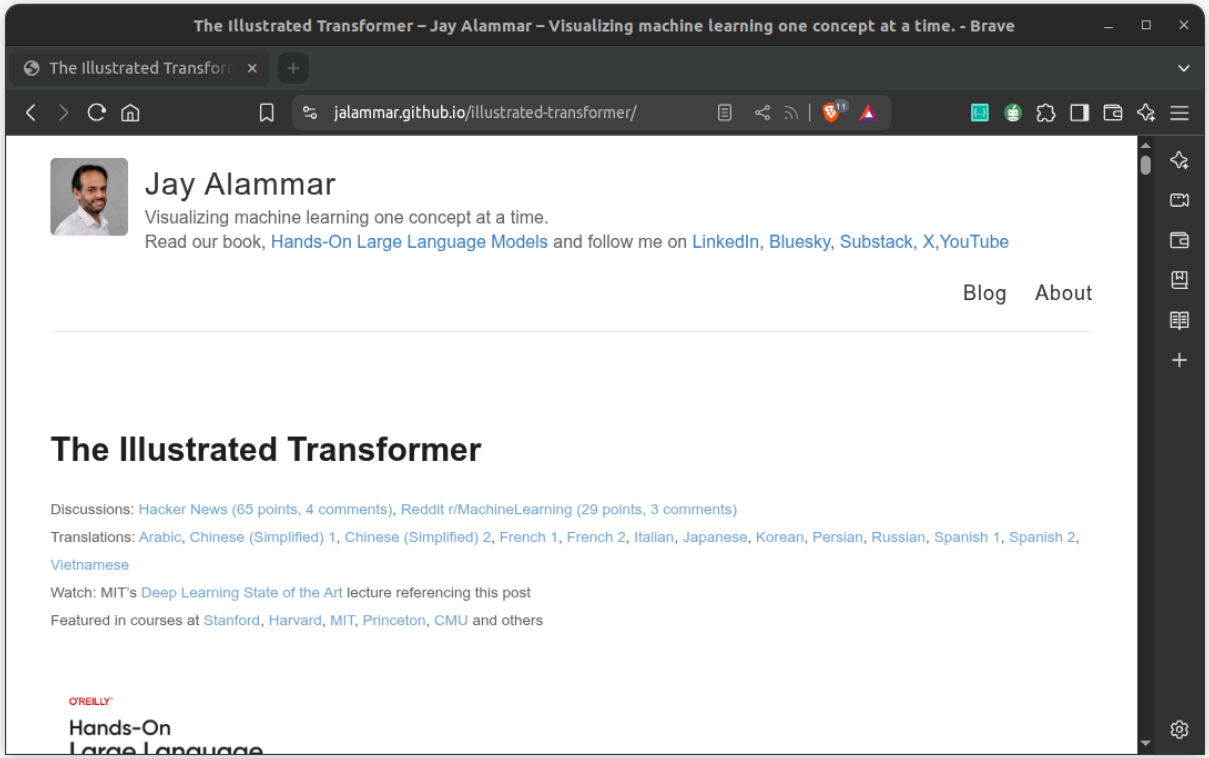


Figure 8: Transformer Illustrated[1].

[1] https://jalammar.github.io/illustrated-transformer/

Task 9: Self-Attention Mechanism Exercise

Consider the sentence "I like to learn" with the following setup:

- Sequence length: 4 words
- Embedding dimension: 5

1. Compute the Query (Q), Key (K), and Value (V) matrices for all words
2. Calculate the attention scores (QK^T) for the word "like" attending to all words
3. Apply softmax to get attention weights for "like"
4. Compute the weighted sum of values for "like"
5. Repeat steps 2-4 for the word "learn"

The learned embeddings for each word are:

E("I") = `[0.1, 0.2, 0.1, -0.3, 0.5]`

E("like") = `[-0.3, 0.4, 0.6, -0.2, 0.1]`

E("to") = `[0.5, -0.1, 0.0, 0.3, 0.2]`

E("learn") = `[0.2, 0.6, -0.4, 0.1, 0.7]`

The weight matrices are:

$$W_Q = \begin{pmatrix} 0.1 & 0.2 & 0.0 & 0.1 & 0.0 \\ 0.0 & 0.1 & 0.2 & 0.0 & 0.1 \\ 0.1 & 0.0 & 0.1 & 0.2 & 0.0 \\ 0.0 & 0.1 & 0.0 & 0.1 & 0.2 \\ 0.2 & 0.0 & 0.1 & 0.0 & 0.1 \end{pmatrix}, W_K = \begin{pmatrix} 0.2 & 0.0 & 0.1 & 0.0 & 0.1 \\ 0.0 & 0.2 & 0.0 & 0.1 & 0.0 \\ 0.1 & 0.0 & 0.2 & 0.0 & 0.1 \\ 0.0 & 0.1 & 0.0 & 0.2 & 0.0 \\ 0.1 & 0.0 & 0.1 & 0.0 & 0.2 \end{pmatrix}, W_V = \begin{pmatrix} 0.1 & 0.0 & 0.2 & 0.0 & 0.1 \\ 0.0 & 0.1 & 0.0 & 0.2 & 0.0 \\ 0.2 & 0.0 & 0.1 & 0.0 & 0.1 \\ 0.0 & 0.2 & 0.0 & 0.1 & 0.0 \\ 0.1 & 0.0 & 0.2 & 0.0 & 0.1 \end{pmatrix}$$

Use a scaling factor of $\sqrt{5} \approx 2.24$ for the attention scores.

Step 1: Compute Q, K, V matrices

First, let's organize our input matrix X (4×5):

$$X = \begin{pmatrix} 0.1 & 0.2 & 0.1 & -0.3 & 0.5 \\ -0.3 & 0.4 & 0.6 & -0.2 & 0.1 \\ 0.5 & -0.1 & 0.0 & 0.3 & 0.2 \\ 0.2 & 0.6 & -0.4 & 0.1 & 0.7 \end{pmatrix}$$

Query Matrix $Q = XW_Q$:

- Q_1 ("I"): [0.12, 0.01, 0.10, 0.00, 0.01]
- Q_2 ("like"): [0.05, −0.04, 0.15, 0.07, 0.01]
- Q_3 ("to"): [0.09, 0.12, 0.00, 0.08, 0.07]
- Q_4 ("learn"): [0.12, 0.11, 0.15, −0.05, 0.15]

Key Matrix $K = XW_K$:

- K_1 ("I"): [0.08, 0.01, 0.08, −0.04, 0.12]
- K_2 ("like"): [0.01, 0.06, 0.10, 0.00, 0.05]
- K_3 ("to"): [0.12, 0.01, 0.07, 0.05, 0.09]
- K_4 ("learn"): [0.07, 0.13, 0.01, 0.08, 0.12]

Value Matrix $V = XW_V$:

- V_1 ("I"): [0.08, −0.04, 0.13, 0.01, 0.07]
- V_2 ("like"): [0.10, 0.00, 0.02, 0.06, 0.04]
- V_3 ("to"): [0.07, 0.05, 0.14, 0.01, 0.07]
- V_4 ("learn"): [0.01, 0.08, 0.14, 0.13, 0.05]

Step 2: Attention scores for "like" (Q_2K^T)

$Q_2 = [0.05, -0.04, 0.15, 0.07, 0.01]$

Attention scores (before scaling):

- $Q_2 \cdot K_1 = 0.05 \times 0.08 + (-0.04) \times 0.01 + 0.15 \times 0.08 + 0.07 \times (-0.04) + 0.01 \times 0.12 = 0.0140$
- $Q_2 \cdot K_2 = 0.05 \times 0.01 + (-0.04) \times 0.06 + 0.15 \times 0.10 + 0.07 \times 0.00 + 0.01 \times 0.05 = 0.0136$
- $Q_2 \cdot K_3 = 0.05 \times 0.12 + (-0.04) \times 0.01 + 0.15 \times 0.07 + 0.07 \times 0.05 + 0.01 \times 0.09 = 0.0205$
- $Q_2 \cdot K_4 = 0.05 \times 0.07 + (-0.04) \times 0.13 + 0.15 \times 0.01 + 0.07 \times 0.08 + 0.01 \times 0.12 = 0.0066$

Scaled attention scores $\left(\div \sqrt{5} \approx \div 2.24\right)$:

$$Q_2K^T = [0.0063, 0.0061, 0.0092, 0.0030]$$

Step 3: Softmax for "like"

Softmax calculation:

- $\exp(0.0063) = 1.0063$
- $\exp(0.0061) = 1.0061$
- $\exp(0.0092) = 1.0092$
- $\exp(0.0030) = 1.0030$

Attention weights ($Sum = 4.0246$):

$$\alpha_2 = [0.2500, 0.2500, 0.2508, 0.2492]$$

Step 4: Weighted sum of values for "like"

$$
\begin{aligned}
\text{Weighted V} &= 0.2500 \times V_1 + 0.2500 \times V_2 + 0.2508 \times V_3 + 0.2492 \times V_4 \\
&= 0.2500 \times [0.08, -0.04, 0.13, 0.01, 0.07] \\
&\quad +0.2500 \times [0.10, 0.00, 0.02, 0.06, 0.04] \\
&\quad +0.2508 \times [0.07, 0.05, 0.14, 0.01, 0.07] \\
&\quad +0.2492 \times [0.01, 0.08, 0.14, 0.13, 0.05]
\end{aligned}
$$

$$\text{Weighted V} = [\mathbf{0.0650}, \mathbf{0.0225}, \mathbf{0.1075}, \mathbf{0.0524}, \mathbf{0.0575}]$$

Step 5: Repeat for "learn" (Q_4)

$Q_4 = [0.12, 0.11, 0.15, -0.05, 0.15]$

Attention scores (before scaling):

- $Q_4 \cdot K_1 = 0.0427$
- $Q_4 \cdot K_2 = 0.0303$
- $Q_4 \cdot K_3 = 0.0370$
- $Q_4 \cdot K_4 = 0.0382$

Scaled attention scores $\left(\div \sqrt{5} \approx \div 2.24\right)$: $[0.0191, 0.0136, 0.0165, 0.0171]$

Attention weights: $\alpha_4 = [0.2506, 0.2492, 0.2500, 0.2501]$

Weighted sum of values for "learn": $= [\mathbf{0.0650}, \mathbf{0.0225}, \mathbf{0.1076}, \mathbf{0.0525}, \mathbf{0.0575}]$

Both "like" and "learn" produce nearly uniform attention weights (0.25 per word), resulting in output vectors that are almost identical. In a trained model, W_Q, W_K, and W_V would learn to amplify meaningful differences.

Attention Mechanisms and Advanced Architectures

Jupyter/attention_transformers.ipynb

6.5 Large Language Models (LLMs)

Large Language Models (**LLMs**) represent a paradigm shift in **AI**, moving beyond traditional **NLP** tasks toward more general language capabilities that increasingly resemble human-like understanding.

They are neural network models trained on vast corpora of text data using self-supervised learning techniques. Most modern **LLMs** are based on the transformer architecture, specifically using variants of the decoder-only design pioneered by **GPT** (**Generative Pre-trained Transformer**) models.

6.5.1 Capabilities and Applications

Modern **LLMs** exhibit remarkable versatility across numerous **NLP** tasks:

- Text generation and creative writing
- Question answering and information retrieval
- Summarization and content transformation
- Code generation and debugging
- Translation between languages
- Reasoning about complex problems
- Instruction following and conversational abilities

6.5.2 Core Characteristics

Scale

Training Data

Emergent Abilities

Transfer Learning

Context Window

6.5.3 Training Paradigm

LLMs follow a training process typically divided into phases:

Pre-training
Fine-tuning
RLHF (Reinforcement Learning from Human Feedback)

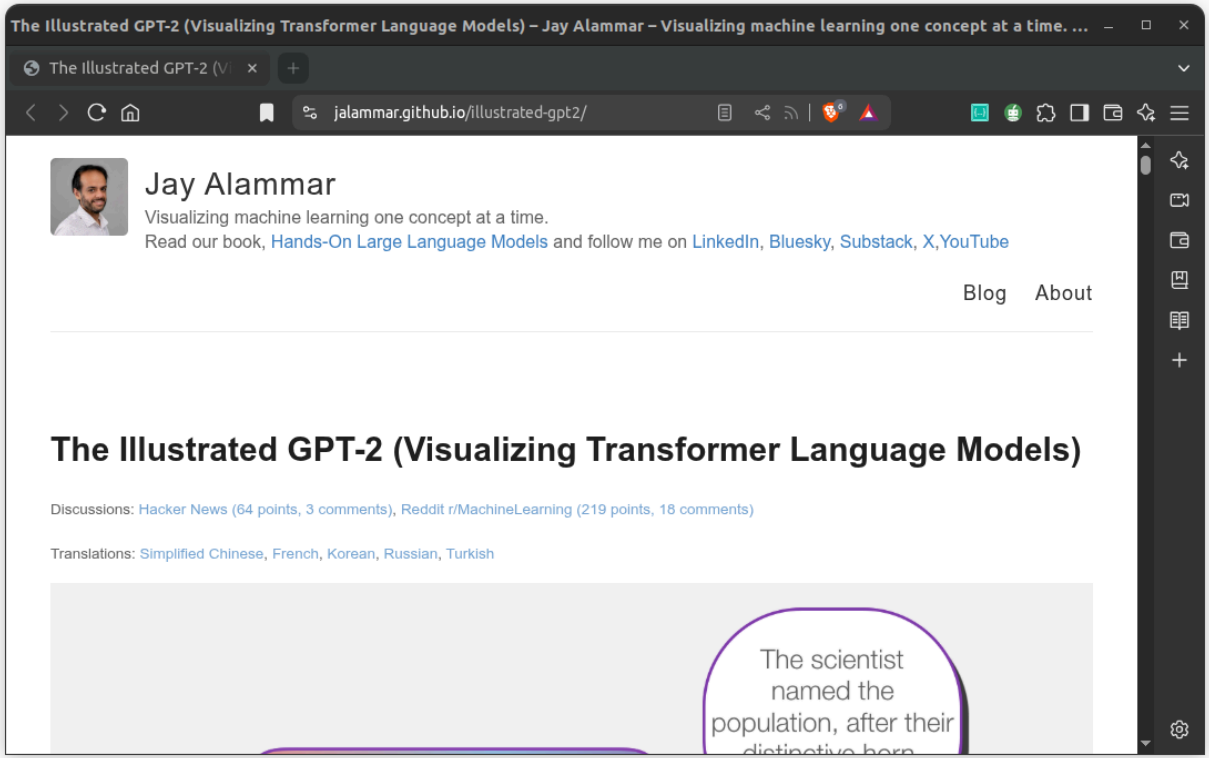


Figure 9: Illustrated GPT2[1].

[1] https://jalammar.github.io/illustrated-gpt2/

6.5.4 Limitations and Challenges

- Hallucinations
- Reasoning limitations
- Training data cutoff
- Computational requirements
- Bias and toxicity
- Black-box nature

6.5.5 Examples of Prominent LLMs

Model	Organization	Year	Notable Features
GPT-4	OpenAI	2023	Multimodal capabilities, advanced reasoning
Claude	Anthropic	2023	Long context window, helpful assistant
Gemini	Google	2023	Strong performance across benchmarks
LLaMA 2	Meta	2023	Open-weights research model
Mistral	Mistral AI	2023	Efficient architecture, strong performance

6.5.6 Model Size Comparison

Model	Parameters (Billions)	Max Context Window	Notes
GPT-2	1.5	1,024	Standard dense transformer
GPT-3	175	2,048	The "Davinci" series foundation
GPT-3.5	175	4,096 – 16,384	16k available in turbo-0613
GPT-4	1,760 (MoE)	8,192 – 128,000	Uses Mixture of Experts (MoE)
Claude 2.1	150+	200,000	Industry leader in context in 2023
PaLM	540	8,192	Google's Pathways architecture
LLaMA 2	7, 13, 70	4,096	Meta's open-weights milestone
Gemini 1.0 Ultra	1,000+	32,768	First model to outperform GPT-4 on MMLU

6.5.7 Future Directions

As **LLMs** continue to evolve, several promising research directions are emerging:

1. **Multimodality**
2. **Efficiency**
3. **Tool use**
4. **Memory systems**
5. **Alignment**

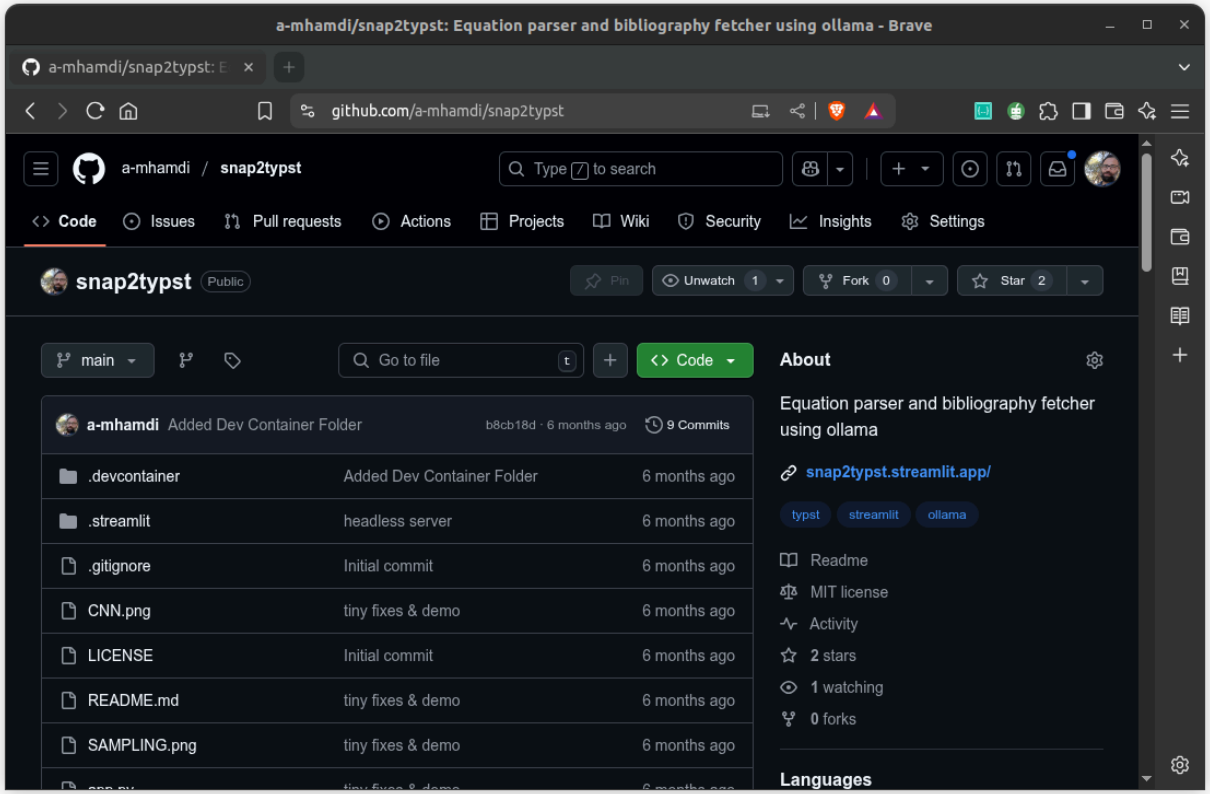


Figure 10: Equation Parser and Bibliography Fetcher for Typst[1].

[1] https://github.com/a-mhamdi/snap2typst

Thank you for your attention

Bibliography

Bibliography

Alammar, J., & Grootendorst, M. (2024). *Hands-on large language models: Language understanding and generation*. O'Reilly Media, Inc.

Brown, P. F., Desouza, P. V., Mercer, R. L., Pietra, V. J. D., & Lai, J. C. (1992). Class-based n-gram models of natural language. *Computational Linguistics*, *18*(4), 467–479.

Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724–1734.

Harris, Z. S. (1954). Distributional structure. *Word*, *10*(2–3), 146–162.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, *9*(8), 1735–1780.

Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). *spaCy: Industrial-strength Natural Language Processing in Python*. https://spacy.io/

Bibliography

Raschka, S. (2025). *Build a large language model (from scratch)*. Manning Publications.

Sparck Jones, K. (1972). *A statistical interpretation of term specificity and its application in retrieval*. *28*(1), 11–21.

Tunstall, L., Werra, L. v., & Wolf, T. (2023). *Natural language processing with transformers: Building language applications with hugging face*. O'Reilly Media.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*.

Řehůřek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, 45–50.